The Session Manager Field Manual

A tab-by-tab operator's guide to Claude Code Session Manager — what each surface is for, the one thing people get wrong about it, and the workflow that makes it pay for itself.

Version 1.7.0 · released 2026-09-02 · documents Session Manager v0.77.0

Contents

1. Getting Started — the 10-minute setup
2. Epics & Sessions — Terminal and Chat are one session
3. File Explorer — one browser, two starting points
4. Scheduler — author once, run around your token window
5. Settings & Scopes — editing the substrate, not the conversation
6. MCP Servers — registering tools, and proving they actually connected
7. Permissions — stop the interruptions, don't disable the guardrail
8. Agent Library — authoring the "who" of an Epic
9. System Prompt — editing the CLI's own house rules, machine-wide
10. Skills — what auto-fires, what you type, and where it lives
11. Plugins — the Installed inspector and the Library catalog
12. Home & Project Brief — the cockpit's dashboard
13. Hooks — edit the CLI's own shell-outs, safely
14. Tag Library — the mission that ships with every Epic
15. Host on Bilko.run — publishing this project's page, gated for real
16. History — where your Claude time and money went
17. Memory Clusters — what an agent has actually been remembering
18. Usage — reading the 5-hour window before it reads you
19. Voice Input — dictate into any session, offline
20. Plans & Tasks — reading what the agent already told you
21. Simple Mode — the chrome-free single-terminal cockpit

Getting Started — the 10-minute setup

Session Manager is a local cockpit for the Claude Code CLI. It does not replace `claude`; it wraps it — giving you a terminal, a scheduler that runs work while you sleep, and roughly two dozen configuration and observability surfaces over the same `~/.claude/` directory the CLI already reads.

1. Install and launch

There is nothing to build. The app ships on npm and runs straight from `npx`:

```
npx claude-code-session-manager@latest
```

First launch recompiles the native terminal backend (`node-pty`) for your Electron runtime, so give it a minute. Linux and macOS only.

If you want the stripped-down version — one terminal, no tabs, no config surface — add `--simple`:

```
npx claude-code-session-manager@latest --simple
```

figure pending capture: app boot, Home tab, empty project list

1

The left nav — every surface lives here. It swaps content with the tab you're on: machine-wide screens on

Home, that project's screens on a project tab. 2 **Open / Start Project**

— the only way a project enters the app. 3

The Almanac footer: connection dot, session (5h) and weekly (7d) usage, active project + branch, live to-dos, app version.

The strip along the top is navigation and nothing else: a **Home** pill for this machine, then one tab per open project. It carries no status and no app branding — those live in the footer.

2. The one idea to internalise: TAB = project, SESSION = unit of work

Almost every mistake new users make comes from missing this. Session Manager has exactly two levels of scope, and they are not interchangeable:

Concept	What it actually is	How many
TAB	A working directory. The cwd *is* the project identity — everything the app stores for a project keys off that path.	One per project. Picking an already-open project re-activates its tab; it never opens a second.
SESSION	One small unit of work inside that project, backed by its own tagged Claude session.	Many per tab. Short-lived by design — one Session per thing you're doing.

> **The rule that follows:** if you want a second conversation in the same project, you want a second *Session*, not a second tab. Tabs are for switching projects.

3. Where your data lives

When a tab first opens on a directory, Session Manager creates one folder inside it:

```
<your-project>/session-manager-operations/
├── prompt-sessions/   # your Sessions — active index + archived Sessions
├── scheduler/         # PRDs, queue, and run state
├── project-brief/     # the Home tab's project brief
└── logs/
```

It is plain JSON and Markdown, it sits inside your repo, and it is meant to be read by humans and committed if you want it committed. Nothing about your project is stored in a cloud service by this app — the marketing page's "local-first" claim is literal.

4. Your first five minutes

1. **Open / Start Project** in the left nav, and pick a repo you actually work in. A native folder picker opens; "New Folder" starts a fresh one.
2. The tab opens on that project, and the sidebar switches to its screens: **Project Home**, **Sessions**, **Scheduler**, **Memory**. Everything runs the same `claude` CLI you already use — same settings, same skills, same MCP servers.
3. Go to **Sessions** and press **+ New Session**. Pick an *Agent* (who works — a persona from your Agent Library) and a *Mission* (feature / bug / discussion), type the outcome you want, and submit.
4. Every session is born *proposed* and costs nothing until you press **Approve & start**. That press sends the composed opening prompt and the session begins.
5. Watch it run — in **Chat** or in **Terminal**. They are two views of the *same* Claude session, so switching never loses context. That's the loop; everything else in this manual is about doing it faster, cheaper, and unattended.

> **Two shortcuts worth knowing on day one.** The row of buttons above the session list is *Hot keys* — each one is an Agent persona scoped to this project, and pressing it opens a pre-composed session in a single click (the **Build** button is the built-in example). And the guided tour that ran on first launch is re-runnable any time: ⌘K / `Ctrl`+K → *Restart guided tour*.

5. What to read next

If you only read two more chapters, read Sessions (so you stop losing context) and Scheduler (so the app starts working while you don't).

Sessions — Terminal and Chat are one session

A Session *is* a tagged Claude session. Not a folder of sessions, not a wrapper around one — a 1:1 relationship. Once that clicks, the two views over it stop being confusing.

One session, two views

Every Session mints exactly one `claudeSessionId` when it is created. Both ways of talking to it attach to that same id:

View	What runs underneath	Best for
Terminal	An interactive PTY: `claude --resume <sessionId>` in the Session's cwd.	Anything needing approvals, interruptions, or you watching closely.
Chat	Headless: `claude -p --resume <sessionId>`, streamed back as a feed.	Long unattended turns, and reading back what happened.

> **Switching views hands the session over — it never forks it.** Attachment is mutually exclusive: exactly one view holds the session at a time. Context continuity across the switch is the whole point, so if you ever see a fresh empty session after switching, that's a bug worth reporting, not expected behaviour.

figure pending capture: Session detail with the Terminal/Chat view toggle visible

1 The view toggle — one session, two renderings. 2

The verbosity dial (1 loudest → 5 quietest). 3

The Session's event chain: prompt → PRD → response.

Creating a Session: two independent choices

The New Session card asks two separate questions, and it is worth being deliberate about both:

1. **Agent — who is working.** A persona from your Agent Library (`~/.claude/agents/*.md`). This also decides which model the session launches with: the persona's own `model:` field wins, falling back to your global default.
2. **Mission — how they work.** One of *feature*, *bug*, or *discussion*. This sets the grounding template the session opens with, and how eagerly the agent should decompose work into scheduled PRDs.

Those two, plus a one-line summary of what's actually grounding the session, compose the opening prompt in a fixed order — **Actor, Input, Mission**, then your own goal text. That's the AIM framework, and it's why a Session tends to start productively instead of spending three turns establishing context.

> Right after Actor, the opening prompt also carries a **Persona notes** section — the selected persona's own markdown body (frontmatter stripped, capped at 6000 chars), not just its one-line description. Claude Code only loads that body itself when a persona runs as a subagent, never for an Epic's own main loop, so this is the only way a persona's central operating rule (e.g. "never implement inline, queue via /develop") actually reaches the session it's supposed to govern.

The "advanced" flip: the grounding board

Flip the New Session card and you get an inventory of what this session will really load — the CLAUDE.md files, skills, MCP servers, and hooks it will discover on disk, plus your git branch and the other Sessions in the project. It is an **inspector, not a control panel**: there is no toggle to make the CLI skip a file it would otherwise find. Read it to understand why an agent behaved a certain way.

Lifecycle: three states, one transition you care about

```
proposed ──(Approve & start, or submitting the first prompt)──▶ active ──▶
```

- **proposed** — reserved but not started. *Nothing has been spent yet.* Every Session is born here.
- **active** — the session is running, authoring PRDs, receiving scheduler updates.
- **completed** — archived to its own JSON file. The session id is dead; resuming mints a new one that traces back.

> **Don't repurpose a Session.** Its title and objective are fixed for the life of its session — that objective *is* the first prompt. Iterate in follow-up messages; when the unit of work changes, start a new Session. The rename action in the row menu is for fixing a typo, not for changing what a Session is about.

Where per-task behaviour belongs (and where it doesn't)

This is the most common architectural mistake users make. Settings — including Permissions, Hooks, MCP Servers — edits *on-disk files that the CLI reads on every single invocation*. A Project-scope Settings change alters the substrate for every current and future Session in that project. It is not per-conversation curation and never was.

You want…	Correct lever
This kind of task to behave differently	A custom **Mission tag** (its prompt template)
A different reviewer/implementer style or model	A custom **Agent persona**
Different permissions, hooks, env vars, MCP servers	**Settings** — genuinely substrate

Actions: a one-click shortcut, not a fourth concept

Every project's Sessions toolbar can carry its own row of **Action** buttons — one per Agent Library persona that declares a project scope. Pressing one is exactly the same act as pressing **+ New Session**: it goes through the same mint → approve → send path, with that persona as the Actor, its first Tag Library mission as the Mission, and the persona's own `action:` instruction as the goal. There is no second creation path and no per-project registry — scope lives on the persona file itself. See Agent Library for how to author one.

Reading a session without drowning

The Chat feed has a five-level density dial, numbered loudest-first: **1 raw**, 2 detail, 3 standard, 4 brief, **5 summary**. Tool cards appear only at levels 1–2. Nothing is ever discarded from the store — raise the level and the byte-exact record comes back with no re-read.

Two things ignore the dial on purpose, and you should know why:

- A turn **asking you something** is never hidden or clamped at any level. Hiding a question strands a run waiting on an answer nobody can see.
- The **in-flight** bubble keeps its tool strip at every level — mid-run it's the only progress signal there is.

File Explorer — one browser, two starting points

File Explorer is the only left-nav destination that shows up on **both** sidebar faces — `projects` is the sole `faces: BOTH` entry in `NAV_ITEMS`. It is a plain file tree plus an editor pane (`ProjectsWorkspace.tsx`), not a second copy of the Terminal or a project switcher in disguise — though it does let you jump projects from inside it.

Which folder it opens to depends on which sidebar you clicked it from

This is the one thing that trips people up, because the tab looks identical either way. The root it browses is resolved from the current sidebar face, not from any setting you pick:

Opened from	Starts at
Home sidebar	Your OS home folder — independent of any project tab you have open elsewhere.
a Project tab's sidebar	That tab's own project directory (its `cwd`) — even if you also have a different project's tab open at the same time.

> If a tab has no project open yet, the Project-face copy shows "No session selected" rather than silently falling back to your home folder — it never guesses at a project you haven't opened.

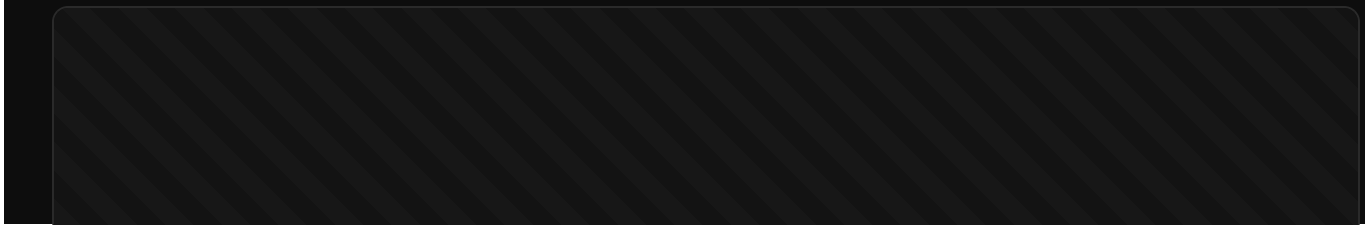


figure pending capture: File Explorer tab, file tree pane on the left with a file selected, editor pane open on the right showing that file's contents

1. The file tree — click a row to open it in the pane on the right; drag the divider to resize.
2. The projects launcher — switch which project's files you're browsing without leaving the tab.
3. The editor pane — dirty-state indicator sits next to the filename.

Two panes: a tree on the left, an editor on the right

The left pane is a collapsible file tree rooted at whichever folder the face rules above resolved. Collapsing it shrinks it down to a thin labeled rail rather than hiding it entirely, and both the split ratio and the collapsed state persist across restarts. The divider between the two panes is draggable, and double-clicking it resets the split back to its default third.

The right pane dispatches by file type: plain text opens in the same document editor other editor-shaped tabs (like System Prompt) use, images and PDFs get their own dedicated preview panes, and anything binary or oversized falls back to a plain info pane rather than trying to render it as text.

Editing: autosave, a dirty indicator, and a close guard

Once a file is open, edits autosave a little over a second after you stop typing — there is no explicit Save button to remember to press. The status line next to the filename tracks this directly: **Editing…** while autosave is pending, **Saving…** mid-write, and **All changes saved** once it lands. Closing a file (or the whole tab) with unsaved changes still pending prompts you before discarding them — the same guard fires whether you're closing one file, the others, or everything at once.

Switching projects without leaving the tab

The projects launcher above the file tree lists every known project, each with a pin toggle (so your most-used projects sort to the top) and an archive action, which asks for confirmation before removing a project from the list. Picking a different project from here swaps the file tree's root directly — it's the fastest way to peek at another project's files without opening a new tab for it.

Scheduler — author once, run around your token window

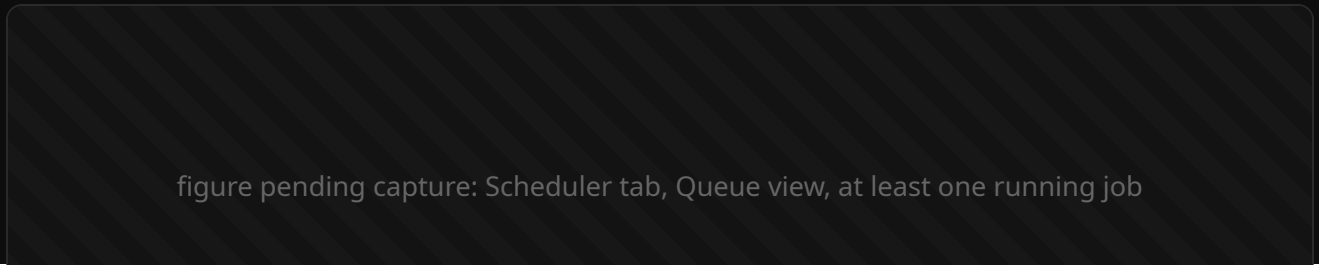
The Scheduler is the feature that pays for the app. You write a PRD, it lands in a queue, and Session Manager runs it as a headless `claude -p` job timed around your plan's rolling 5-hour token window — auto-pausing when you're rate-limited and resuming the moment the window resets.

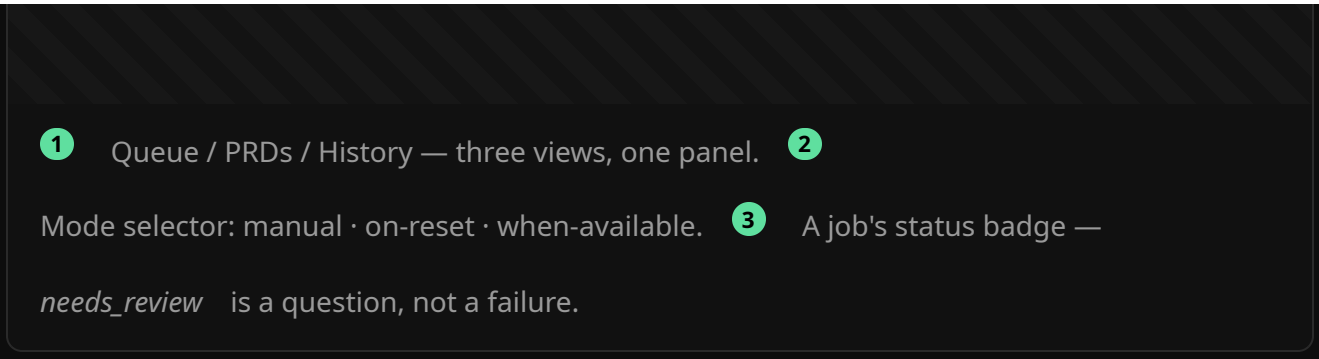
What the Scheduler is a browser over

It is a viewer and operator surface on one hierarchy, scoped to the active tab's project by default ("All projects" is the explicit escape hatch):

```
TAB (project)
└── SESSION (unit of work)
    └── PRD (one scheduled job)
```

On disk, PRDs live at `<project>/session-manager-operations/scheduler/epics/<epic-id>/prds/`, with the queue and history sharded alongside in `scheduler/state/`. Every PRD belongs to a Session — there is no such thing as a free-floating PRD.





1 Queue / PRDs / History — three views, one panel. 2 Mode selector: manual · on-reset · when-available. 3 A job's status badge — *needs_review* is a question, not a failure.

The three run modes

Mode	Behaviour	Use when
`manual`	Nothing runs until you press go.	You're debugging the queue itself.
`on-reset`	Fires at each 5-hour window reset.	You want a predictable burst, not a trickle.
`when-available`	Polls your billing usage every 60 s and runs whenever there's headroom. **The default.**	Almost always.

Concurrency: one pool, machine-wide — with three inner gates

Scheduler jobs and Chat runs draw from the *same* pool of concurrent `claude -p` sessions (default 5, adjustable from the Home tab). Underneath that outer pool sit three more predicates that decide, tick by tick, which pending jobs actually launch — none of them add a second pool; they only withhold launches from the one that exists:

- **Memory gate.** A job won't start if the machine doesn't have headroom for it. Machine-aware rather than a hardcoded job count, which is what makes running five parallel agents on a laptop survivable.
- **Per-project cap.** No single project can claim the whole pool — by default each project gets at most 2 concurrently running jobs, so one busy project can't starve

every other open project of a slot.
- **CPU-load gate.** A job is held back when the 1-minute load average per core exceeds a threshold (default 0.85), even if slots and memory are free. This catches contention a memory gate can't see — e.g. several CPU-heavy test batteries running at once slowly starving each other of wall-clock, each one then overrunning its own timeout.

On top of the per-project cap, the scheduler also enforces **cross-project fairness**: when there's more pending work than free slots, slots are handed out round-robin across projects that have pending jobs — one job per project per pass — so a single project can never drain every free slot in one tick while another project with eligible work gets none. If a project's oldest pending job has waited past a threshold while some other project keeps getting slots, that project is flagged as starved so it gets attention rather than silently waiting indefinitely.

Writing a PRD that actually finishes

Two real incidents shaped every rule here, and both are worth internalising:

The poll-hang

A PRD used `until $(curl … | jq .uptime)` to wait for a deploy. The condition was unsatisfiable — that deploy type never restarts the API — so the job spun for **2 h 47 m** until the watchdog killed it.

Rule: never write an unbounded wait. Every loop gets a maximum iteration count and an explicit give-up branch.

The post-AC overrun

A PRD declared success, then launched a fixture generator over 50,000 seeds that appeared nowhere in its acceptance criteria. It ran **2 h 44 m** before a human killed it.

Rule: the acceptance criteria are the *ceiling*, not the floor. When the AC are met, stop.

The checklist

1. One PRD, one outcome. If you can't state the outcome in a sentence, it's two PRDs.
2. Acceptance criteria must be checkable **headlessly**. "Launch the app and click through the tabs" cannot be verified by a `claude -p` job — it has no GUI and will be SIGTERM'd.
3. Every loop is bounded. Every wait has a timeout.
4. Ordering comes from `dependsOn` — nothing else gates a batch.
5. Never reuse a PRD number.

> **PRD authoring is API-only.** The `scheduler_create_prd` MCP tool is the sole sanctioned way to write a PRD file — a hook blocks any other tool from writing directly into a project's `scheduler/epics/<epic-id>/prds/` folder. It validates frontmatter, atomically allocates the PRD number, and enforces that every PRD joins an existing Session rather than floating free. You will never need to hand-write a PRD file yourself.

A PRD's `agentType` field picks who runs it — a persona name, defaulting to `dev-lead` — and it drives more than the prompt: the scheduler resolves it to that persona's own system prompt *and* model, so a PRD authored for a review-heavy persona runs under that persona's chosen model without you specifying `--model` yourself. If the named persona no longer exists, the job still runs — it just falls back to no persona and the default model, logged once rather than parking or failing.

When a job parks in `needs_review`

`needs_review` is **a question, not a result**. When a job lands there, Session Manager writes a deterministic root-cause report next to that run's own log and appends it as a

response on the Session that authored the PRD — so the question comes back to the conversation that asked it, not to a global inbox you'll never check.

This is also the strongest argument for writing tight PRDs: the clearer the PRD, the less there is to route back to you.

Self-healing you don't have to think about

- **Boot reconciliation** — on startup, jobs whose processes died are reaped and classified (success / timeout / error). A still-alive orphan is terminated gracefully first so its log isn't misread mid-write.
- **Dead-process reaper** — periodic PID-liveness checks catch hung jobs.
- **Supervisor** — every 15 minutes, a probe per running job; it kills a stuck poll-loop's shell without killing the agent that spawned it.
- **Definition-of-done gate** — when the queue drains, completed PRDs are re-verified against their own acceptance criteria and a report is written. Idempotent, so a re-drain over the same set is a no-op.
- **External watchdog** — an out-of-process timer relaunches the app if its heartbeat goes stale while jobs are outstanding.
- **Launch circuit breaker** — a run that 400s before it ever got a turn (a bad CLI/model flag combination, not your PRD) is recognized as a launch failure, not a failed PRD: the job resets to pending and a per-persona breaker opens, backs off, and probes itself back closed once launches succeed again — so one bad CLI update can't burn through every queued job logging the same failure.

> All of it exists so that "queue it and go to bed" is a real workflow rather than an aspiration.

Settings & Scopes — editing the substrate, not the conversation

The Settings tab is a scoped editor over the three files the `claude` CLI itself reads on every launch. It looks like a normal per-project config screen, and

that similarity is exactly what causes the most common mistake in this app: reaching for Settings when what you actually wanted was a Tag or an Agent persona. This chapter covers the scope model, the one field that legitimately crosses the line into per-Session territory, and how to stop yourself before you edit the wrong layer.

The three scopes

Settings, Permissions, Hooks, MCP Servers and Skills all share one shape — **ScopeSwitcher + SaveBar + JsonEditor** — over the same three on-disk files, resolved by `lib/scopes.ts`'s `SETTINGS_SCOPES`:

Scope	File	Applies to
User	`~/.claude/settings.json`	Every project on the machine.
Project	`<cwd>/.claude/settings.json`	This project only — meant to be checked into git and shared with collaborators.
Local	`<cwd>/.claude/settings.local.json`	This project, this machine only — gitignored, for machine-specific overrides.

Permissions is the same three files under the hood — it's a structured UI over the `permissions` subtree of the identical `settings.json`, not a separate file. The tab defaults to **Project** scope when you're looking at an open project tab and **User** scope on the Home face, but that's a starting point, not a boundary — you can switch scope at any time, and the toolbar always tells you which file is active (`overrides → <path>` in Guided view, the raw path in Tree/Raw view).

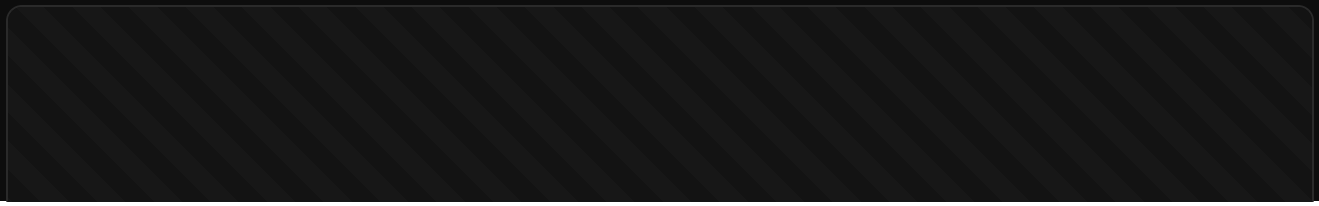


figure pending capture: Settings tab, Guided view, ScopeSwitcher showing User/Project/Local with one dirty scope

1. Guided / Tree / Raw — three views of the same merged config.
2. Scope switcher — User, Project, Local, each annotated with exists/dirty state.
3. The active file path, so you always know exactly what you're about to save.

Guided view (`EffectiveCards`) and Tree view (`EffectiveTree`) both show the **merged, effective** config across all three scopes — editing a value there writes an override into whichever scope is currently selected. Raw view is the literal JSON of the one active scope's file, through Monaco with schemastore.org validation. Switching to Raw and saving directly is the fastest path when you already know the exact key you want; Guided is the safer path when you're not sure what's already set somewhere else.

Settings is machine/project substrate — not per-conversation curation

> **The rule that matters most in this chapter:** every file Settings edits is a file the `claude` CLI reads fresh on *every single invocation it launches* — every Session, every Terminal tab, every scheduled PRD job, in that project (Project/Local scope) or on the machine (User scope). Settings is not scoped to one conversation, one Session, or one task. There is no "just for this chat" checkbox, because the file underneath has no concept of "this chat."

That means a Project-scope edit changes behaviour for **every current and future Session in that project**, and a User-scope edit changes it for **every project on the machine** — retroactively, for sessions that already exist and ones you haven't created yet. If you only wanted one Session to behave differently, you just changed the wrong number of things.

You want…	Wrong lever	Right lever
"This one Session should follow a different process / think about the problem differently"	Editing Settings for the duration of the task, then editing it back	A Tag (Tag Library, `initialPromptTemplate`) or an Agent persona (Agent Library) chosen when you create the Session
"This kind of task should always run with a particular persona/model"	Flipping the Settings model default before starting, back after	An Agent persona with its own `model` frontmatter field, selected as that Session's Agent
"Every project on this machine should allow a tool / set an env var / register an MCP server"	—	Settings, User scope. This one *is* the right lever — it's genuinely machine-wide, task-independent config.
"This project's hooks/permissions/MCP servers should be the same for anyone who clones the repo"	—	Settings, Project scope, committed to git. Also correct — it's project-wide substrate, not task curation.

Put another way: Settings holds what's legitimately identical *regardless of what task you're about to run* — permissions, hooks, MCP servers, env vars, output style. If a value should differ depending on *what kind of work* a Session is doing, it doesn't belong in Settings at all; it belongs on the Session's Tag or Agent, chosen fresh at that Session's creation and folded into its opening prompt.

The one field that's carved out per-Session: `model`

There is exactly one exception, and it's deliberately narrow. An Agent persona (`~/.claude/agents/<name>.md`, edited from Agent Library) can carry its own `model` frontmatter field. When a Session is created with that persona as its Agent, `src/main/lib/agentModelResolve.cjs`'s `resolveEpicModel` looks up the persona's `model` field first — and only falls back to Settings' "Model & Reasoning" default (`sonnet`) when the persona has no `model` set, or it's explicitly `'inherit'`. This is the same resolution both views of that Session use: Chat (`chatRunner.cjs`) and Terminal (`EpicTerminalPane.tsx`'s own mirrored resolver) agree on which model a Session launches with, because they're two views over the same session.

Every *other* field in Settings' "Model & Reasoning" group — `effortLevel`, `fastMode`, `alwaysThinkingEnabled`, and the rest — has no per-Session override today. Editing them changes the default for every Session that doesn't specify otherwise, and there's currently no per-Session escape hatch for them the way there is for `model`.

> **Rule of thumb:** if you're about to open Settings to change how one Session behaves, stop — open Agent Library or Tag Library instead. If you're about to open Agent Library or Tag Library to change something for every project on the machine, stop — that's Settings, User scope.

What lives outside the three scoped files

Settings' toolbar has two more views that aren't scoped at all — **Telemetry** and **Session Manager** — visible only on the Home face, because both are genuinely machine-wide with no cwd or scope input:

- **Telemetry** (`SettingsTelemetry.tsx`) — opt-in OTEL export of classified transcript events, persisted to `~/.config/session-manager/otel.json`. Not a Claude Code file at all; this is Session Manager's own instrumentation, off by default. An "Include content" toggle controls whether tool inputs/plan text/agent prompts are exported, mirroring upstream Claude Code's own `OTEL_LOG_USER_PROMPTS` opt-in — structural metadata (tool names, todo counts, usage tokens) goes out either way.

- **Session Manager** (`SettingsAppPrefs.tsx`) — native app preferences stored in `localStorage` , applied immediately with no save/revert cycle. Today this is one setting: the default model for a *raw* interactive session (one opened without going through a Session at all) — separate from `resolveEpicModel` 's persona-first resolution, since a raw session has no Agent to read a `model` field from.

Both switch away automatically if you flip from the Home face to a Project tab while one is open — they have nothing project-scoped to show, so the toolbar never displays a tab that no longer applies.

Reading the merged view before you touch a scope

Guided and Tree views compute one merged object across User → Project → Local (precedence rising in that order, plus Managed and CLI-flag layers the app doesn't edit) via `mergeScopes` . Before overriding a key, check *which* scope already holds it — the Guided view's reset-to-default button targets the most-specific scope currently holding that key, not blindly the scope you happen to have selected. Writing an override at User scope when Project scope already sets the same key does nothing observable until someone deletes the Project-scope value — a quiet trap if you're debugging "why didn't my setting take effect."

> The schema-reference link in the toolbar (`code.claude.com/docs/en/settings`) is the authoritative field list. Monaco's own `jsonDefaults` validates against schemastore.org as you type in Raw view — there's no hand-rolled validation layer to fight with.

MCP Servers — registering tools, and proving they actually connected

The MCP Servers tab edits the same JSON the `claude` CLI reads on every launch — `~/.claude.json` at User scope, `.mcp.json` at Project scope — and adds one thing raw JSON editing can't give you: a live probe that tells you

whether a registered server actually connected, before you build a Session that assumes its tools exist.

Where a server definition lives

There is no session-manager-owned server registry. The tab is a scoped editor over two files, switched with the same **User / Project** scope switcher used elsewhere in the app:

Scope	File	Applies to
User	`~/.claude.json`	Every project, every session, on this machine.
Project	`<project cwd>/.mcp.json`	This project only — checked into the repo if you commit it.

Both files share the same shape: a top-level `mcpServers` object keyed by server name. This repo's own `.mcp.json` is a real example — it registers two servers, `session-manager-scheduler` (`node scripts/scheduler-mcp-server.cjs`, a wrapper that exposes the admin API's `scheduler_list_jobs` / `scheduler_reset_job` tools) and `bilko-host` (`node` against a path in the sibling `Bilko` repo) — both plain `stdio` servers with no `type` field set, since `stdio` is the default transport.

> On first launch, the app also registers `session-manager-scheduler` at **User** scope automatically — a project's own `.mcp.json` only reaches that project, so without a User-scope registration every other project on the machine silently lacked `scheduler_create_prd` and agents fell back to hand-writing PRDs or implementing inline. This is a one-shot, idempotent seed: it never overwrites an existing registration (manual or otherwise), and a deliberate `claude mcp remove` stays removed. Opt out with `SM_SEED_SCHEDULER_MCP_DISABLE=1`.

figure pending capture: MCP Servers tab, User scope, several servers with mixed connection states

1 Scope switcher — User vs Project, same file the CLI reads. 2

Status dot per server: live probe result, not a guess from the config alone. 3

⟳ **test connections** — re-runs the probe on demand.

The field you'll actually touch, per server

Field	Meaning
`type`	`stdio` (default) · `http` · `streamable-http` · `sse` (deprecated — the editor flags it) · `ws`.
`command` / `args`	For `stdio` servers — the process the CLI spawns.
`url` / `headers`	For the HTTP-family transports.
`env`	Environment variables passed to a `stdio` server's process.
`alwaysLoad`	Bypasses tool-search deferral (Claude Code v2.1.121+) — the server's tools are always in context instead of loaded on demand.
`enabled`	Session-Manager-editable toggle: unchecking it sets `enabled: false` so the server is skipped in new sessions without deleting its definition.
`disabledTools`	Per-server tool denylist — same list `/mcp` surfaces inside a Claude session.
`permissions.tools`	`inherit` (default) · `allow` all · `deny` all · an explicit allowlist array.

Field	Meaning
`description`	Not a Claude Code config field — a Session-Manager-only purpose note, so a server list of ten names isn't ten mystery strings later.

> One name is off-limits: Claude Code refuses to load a server named `workspace` (reserved as of CLI v2.1.128). The editor blocks it with a hard warning. `ide` and `tasks` aren't reserved, just commonly used by other tooling — the editor flags those as a caution, not a block.

The common mistake: registering a server and trusting it works

Adding an entry to `mcpServers` only proves the JSON is well-formed — it says nothing about whether the CLI can actually reach the server. A `stdio` command that's misspelled, an HTTP server that needs an OAuth flow you haven't completed, or a plugin-provided server still waiting on your approval all look identical in the raw file: present, and untested. The mistake is starting a Session that assumes a server's tools are available, discovering three turns in that the agent silently has none of them.

The workflow that pays off

1. Add or edit the server entry, then **Save** — it writes straight to `~/.claude.json` or `.mcp.json`, whichever scope is active.
2. Press ⟳ **test connections**. This shells out to `claude mcp list` — a plain subcommand, no model call — and matches each configured server name against what came back.
3. Read the status dot before relying on the server anywhere:

Status	What it means
Ready	Connected. Safe to build a Session around its tools.
Failed	The CLI tried and couldn't connect — check the command/URL first.
Needs auth	An OAuth-style server waiting on you to authorize it.
Pending approval	A plugin-registered server waiting on a trust decision.
Not checked	No probe result yet for this name — press test connections, don't assume.

4. Only then start (or resume) the Session that depends on this server's tools. A session started before the server was reachable won't retroactively pick it up — the tools are discovered at launch, not mid-conversation.

The probe has a 30-second ceiling and coalesces concurrent requests onto one in-flight `claude mcp list` call, so mashing the button doesn't fan out into parallel CLI processes — but it also means the result can lag a few seconds behind a config change you just saved. Re-test after every edit that touches connectivity (command, URL, env, headers), not just once at the start of a session.

The Library view: catalog, not installer state

The view switcher next to the scope switcher flips between **Installed** (the editor above) and **Library** — a browsable catalog of known MCP servers you can add with one click. The Library only ever reads `~/.claude.json`'s `mcpServers` keys to mark which catalog entries are already installed; it has no separate connection state of its own. Installing from the Library still lands you back in the same "unprobed until you test it" situation above — a catalog entry existing doesn't mean the server it describes is reachable on your machine.

Permissions — stop the interruptions, don't disable the guardrail

The Permissions tab edits the same `permissions` block the `claude` CLI itself reads out of `settings.json` — allow, deny, and ask rules, plus a default mode and a list of additional directories. It is a scoped editor over real config, not a session-manager-only feature: every rule you write here is exactly what the CLI enforces on its own, with or without this app open.

What it's actually for

The honest framing is **reducing prompt interruptions**, not building a security boundary. Claude Code already asks before anything destructive by default — the Permissions tab exists so you stop re-approving the same handful of safe, read-only commands every session. `deny` rules are a real hard stop and worth using for genuinely dangerous patterns, but the tab's daily value is in `allow`: tell the CLI once that `Bash(ls:*)` or `Read(~/.claude/**)` never needs a prompt, and it stops asking.

The three lists, and the mode

Field	What it does
`allow`	Rules that auto-approve without a prompt — the one you'll edit most.
`deny`	Rules that hard-block, no prompt, no override at run time.
`ask`	Rules that force an explicit prompt even if something else would allow it.

Field	What it does
`defaultMode`	One of `default`, `acceptEdits`, `plan`, or `bypassPermissions` — the CLI's baseline posture before any rule list is consulted.
`additionalDirectories`	Extra paths the session may touch beyond its own working directory.

Rules use the CLI's own syntax (`Tool(pattern)`, e.g. `Bash(ls:*)`, `Write(/etc/**)`) — the tab links straight to the CLI's own rule-syntax docs rather than re-documenting it, and this manual won't either. Get the pattern right there, not by guessing here.

figure pending capture: Permissions tab, Rules view, Project scope, an allow rule being added

1 Effective / Rules / Presets — three views over the same scope. 2

Scope switcher: User / Project / Local. 3 Allow / Deny / Ask — add a rule, hit Enter.

Effective vs. Rules — read before you write

The tab opens on **Effective**, not Rules, and that ordering is deliberate: it shows the merged result of every scope stacked together, so you can see what the CLI will actually do before you go add a rule that's already covered — or that a higher scope already blocks. Switching to **Rules** edits the raw list for whichever single scope is selected. From Effective, clicking an inherited value can push an override straight into the active scope instead of retyping it by hand.

Scope: the mistake people make

Permissions has the same three scopes as Settings — **User** (`~/.claude/` , every project on the machine), **Project** (`<cwd>/.claude/settings.json` , checked into git, shared with the team), **Local** (`<cwd>/.claude/settings.local.json` , gitignored, per-machine). The common mistake is adding a personal convenience rule — allowlisting a command you happen to trust on your own machine — at **Project** scope, where it ships to every teammate who pulls the repo and silently changes what their sessions auto-approve too.

> **The workflow that pays off:** allowlist genuinely safe, read-only commands (status checks, linters, `git diff` , read-only greps) at **User** scope so they stop interrupting you everywhere. Reserve **Project** scope for rules the whole team should share — a build step everyone runs, a deploy command the team has agreed is safe. Keep anything machine-specific or experimental in **Local**, where it never leaves your disk.

Presets

The Home face adds a fourth view, **Presets** — a static reference library of rule bundles, distinct from Rules/Effective in that it has no scope or cwd input of its own. It isn't offered on the Project face; landing there with Presets selected bounces you back to Effective. Use it to see or copy a starting bundle, not to edit a live scope directly.

The default-mode buttons

Above the rule lists sits one more control: `defaultMode` , a single choice among `default` , `acceptEdits` , `plan` , and `bypassPermissions` . This is the CLI's baseline stance before any allow/deny/ask rule is even consulted — the widest, bluntest lever on the tab. Set it at the scope where it belongs: a personal `acceptEdits` habit belongs in User or Local, not pushed into the Project file everyone inherits.

Automating the allowlist

You don't have to build the allow list by hand. The `fewer-permission-prompts` skill scans your own transcripts for the read-only Bash and MCP calls you keep re-approving, and writes a prioritized allowlist straight into `.claude/settings.json` — the same file this tab edits. Run it, then open Permissions → Rules to review what landed before you save; it's a generator for the list, not a replacement for looking at it.

1. Work normally for a session or two — approve prompts as they come.
2. Run the `fewer-permission-prompts` skill to mine those approvals into a candidate allowlist.
3. Open the Permissions tab, switch to **Rules** at the scope the skill wrote to, and read what it added.
4. Move anything personal-only down to Local; leave anything genuinely team-safe at Project.
5. Save. The next session with a matching command runs without a prompt.

> Permissions is a friction dial, not a lockbox. If you actually need to stop an agent from doing something, write a `deny` rule — but most of what people reach for this tab to do is just "stop asking me," and `allow` at the right scope is the whole answer.

Agent Library — authoring the "who" of a Session

Agent Library is a full CRUD editor over `~/.claude/agents/*.md` — the same persona files the `claude` CLI already reads. It is not a viewer bolted onto the filesystem; New, Duplicate, Save, Revert, and Delete all write straight through to disk, and every persona you author here becomes a pickable *Agent* in the New Session card's "who is working" column.

What a persona actually controls

A Session is always grounded by two independent choices: an **Agent** (this page — the "who") and a **Mission** tag (Tag Library — the "what"). Picking a persona in the New Session card does three concrete things, and only three:

Field	Where it lands	Notes
`description`	Folded into the Session's opening prompt as an Actor framing line, before the grounding summary and the Mission template.	Read once, at Session creation. Editing the persona afterward does not rewrite an existing Session's opening prompt.
`model`	Sets the `--model` the Session launches with, for **both** the Terminal view and the headless Chat view.	Only fires when set to something other than `inherit`. Otherwise each view falls back to its own default (Chat: `sonnet`; Terminal: the raw-session-model toggle).
`tools`, `color`, `tags`, definition body	Written verbatim into the persona's frontmatter/body — read by the `claude` CLI or Tag Library, not re-interpreted by session-manager.	`tags` is a session-manager-only extension; the CLI itself never reads it.

> Beyond those, `agentType` stays display-only. Picking a persona does not change which `claude` binary spawns, and it does not swap out permissions, hooks, or MCP servers — that substrate is Settings, not Agent Library. See the domain-model rule in this manual's project reference: if you want a task to run under different settings, that's a Settings edit; if you want it to run as a different *persona*, that's this page.

figure pending capture: Agent Library tab, a persona selected in the list, detail pane open showing model/tools/tags fields

1 The persona list — filterable, each row shows its color swatch and override count. 2 **default model** — sets `--model` for this persona's Sessions. 3 **tags** — the many-to-many link to Tag Library, editable from either side.

The fields, in the order you'll fill them

1. **name** — the filename on disk (`~/.claude/agents/<name>.md`). Must be lowercase and hyphenated (`^[a-z][a-z0-9-]*$`); the app rejects anything else before it ever hits disk.
2. **description** — shown in the persona list and in every Agent picker. This is the line that gets folded into a Session's opening prompt, so write it as a sentence about what this persona does, not a label.
3. **default model** — one of `inherit` / `haiku` / `sonnet` / `opus` . Leave it `inherit` only if you genuinely want this persona to ride whatever the launching view's own default is; otherwise pin it.
4. **tools** — a checklist of the tools this persona is granted (`Read` , `Grep` , `Glob` , `Bash` , `Write` , `Edit` , `WebFetch` , `WebSearch` , `Task`). Anything not ticked is refused at call time by the CLI, not by session-manager.
5. **color** — cosmetic only; shows as a swatch dot in the persona list.
6. **tags** — which Tag Library missions this persona is associated with. Purely a many-to-many bookkeeping field (see below), not a restriction on which Mission a Session can pair it with.
7. **definition** — the system prompt body this persona runs under.

Model resolution: how `--model` actually gets picked

Both launch paths — `chatRunner.cjs`'s headless Chat run and `EpicTerminalPane.tsx`'s interactive Terminal — resolve the same way, mirrored so the two views of one Session never disagree on which model they're running:

1. Look up the Session's `agentType` (the persona it was created with).
2. Read that persona's own `model` frontmatter field from `~/.claude/agents/<agentType>.md`.
3. If it's set and isn't `inherit`, use it.
4. Otherwise fall back — Chat falls back to the hardcoded `sonnet` floor; Terminal falls back to its own raw-session-model toggle.

Any failure in that chain — no matching Session, no persona file, an unreadable file — falls back silently rather than throwing. The lookup never blocks a launch; the worst case is you get the fallback model instead of the one you thought you set.

Tags: the same relationship, editable from two pages

A persona's `tags` field records which of Tag Library's missions it's associated with. This is a genuine many-to-many relationship with one source of truth: assigning a tag from this page's chip row and assigning an agent from Tag Library's own detail pane both write through the identical `savePersona` call, to the identical frontmatter line. There is no second copy to drift.

Turning a persona into an Action button

Any persona can carry a project scope, which promotes it to its own button in that project's Sessions toolbar — a shortcut around the New Session card, never a second creation path. Three frontmatter fields, all editable from this page's detail pane:

Field	What it does
`projects`	Which project cwds show this Action, or `*` for every project.
`action`	The opening instruction — becomes the Session's goal, the way your own typed goal does from the New Session card.
`actionLabel`	The button's text. Defaults to the persona name if left blank.

> Pressing an Action button composes a Session exactly the way New Session does: this persona is the Actor, its first Tag Library mission is the Mission, and `action` is the goal text — same mint → approve → send path, same single-creator rule. The one built-in exception is the `builder` persona, which is deliberately excluded from the dynamic Action list because its own **Run Build** button already covers publish-target gating and in-flight dedup that a generic Action can't express.

Project overrides — the mistake that costs the most time

Claude Code itself resolves a project-local agent file (`<project-cwd>/.claude/agents/<name>.md`) ahead of the global one at `~/.claude/agents/<name>.md`, for any project that has one. Agent Library shows you this directly: a persona row with an **override** badge names every currently-open project shadowing it, and the detail pane's "project overrides" field lists them with a one-click **×** to drop a project's local copy.

Editing the global persona and wondering why nothing changed

You open a persona, tighten its tool grants or rewrite its definition, save — and the next Session launched from a specific project behaves exactly as before. The project has its own `.claude/agents/<name>.md` silently winning every time, and Agent Library's edit form never touches it — this page only creates and edits the *global* copy.

Rule: before editing a misbehaving persona, check the "project overrides" field for that name. If a project you're testing against is listed, either edit that project's local file directly or drop the override from here first.

> Agent Library itself can only create the global definition — it has no per-tab cwd context of its own, so there is no "create an override" button here. Overrides are created by Claude Code's own file-discovery, or by hand; this page can only read and delete them.

The workflow that pays off: focused personas instead of repeated instructions

The failure mode this page exists to fix is retyping the same paragraph of constraints into every prompt — "only read, never edit," "use opus for this, it needs judgement," "stick to these three tools." Every one of those is a one-time persona edit instead of a recurring tax on your attention:

- A read-only research persona: tools trimmed to `Read` / `Grep` / `Glob`, model left cheap. Reuse it for every "just look and report" Session without re-explaining the constraint each time.
- A judgement-call persona: `model` pinned to `opus`, tagged against the Mission it's meant to pair with in Tag Library, so it surfaces as the suggested default there.
- **Duplicate** an existing persona that's 90% right rather than authoring from scratch — it clones the full draft (name suffixed `-copy`) so you're editing, not retyping.

The description field is the lever most people under-use: because it's folded into the Session's opening prompt ahead of the Mission template, a well-written description does real framing work — it is read by the model as the first line of context, not just displayed in a list.

Deleting a persona

Delete removes the global `.md` file — it asks for confirmation once (inline, not a modal) and is otherwise unconditional: there is no "in use by a running Session" guard on this action, so double-check the "project overrides" list and any Sessions you know reference this persona by name before confirming. A persona referenced by `agentType` on an existing Session that no longer resolves simply falls back through the same chain described above — the Session keeps running, just under the fallback model instead of a deleted persona's pinned one.

System Prompt — editing the CLI's own house rules, machine-wide

System Prompt is a Home-face-only Configure tab that edits one real file: `~/.claude/CLAUDE.md`, the instructions the `claude` CLI loads into every session it starts, on this machine, in every project. It is not a preview or a copy — saving here writes the actual file the CLI reads next time it launches.

> **User system prompt.** This is the real `~/.claude/CLAUDE.md` on disk — the instructions the `claude` CLI loads into every session it starts on this machine, in every project. A project's own `CLAUDE.md` / `CLAUDE.local.md` are read from that project's directory when a session launches there; they are not edited here.

Why there's no scope switcher here

Other editor-shaped tabs in this app carry a User / Project / Local scope switcher. System Prompt deliberately doesn't: it's reachable only from the Home face, which has no project context of its own, so a Project or Local scope selector here would either dead-end or silently point at whichever project happened to own some other open tab. The page reads and writes exactly one scope, always — `user` — and that choice doesn't change if you switch to a Project face with a tab already open.

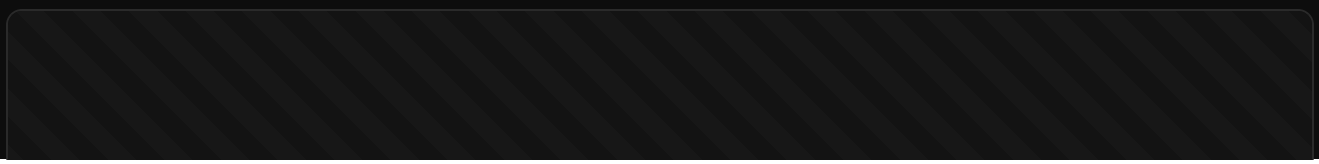


figure pending capture: System Prompt tab, referenced-files rail on the left listing the root CLAUDE.md plus its @-imports, main editor pane open on the right, save bar at the bottom

1 The referenced-files rail — the root file plus every file it `@` -imports. 2

The editor pane — Edit / Preview / Split, same chrome the Editor tab uses. 3

The save bar — explicit Save / Revert, with a dirty-state indicator.

Imports are editable, not just previewed

If your `~/.claude/CLAUDE.md` uses `@path/to/file.md`-style imports — the way this very project's own instructions are composed — the left rail lists every one of them alongside the root file. Clicking an imported file opens it in the same editor pane, and it's genuinely editable: whatever you save there is literally part of the prompt the CLI loads, not a read-only preview of it.

The one exception is an import that doesn't resolve to a readable file. That row still shows up in the rail, but opens forced read-only with a warning banner:

> "This import does not resolve to a readable file — the CLI will skip it."

Saving

Unlike File Explorer's editor pane, System Prompt does not autosave — a save bar at the bottom carries explicit **Save** and **Revert** actions, tracking dirty state and the last-saved time so you always know whether what's on screen matches what's on disk. Saving while an imported file is open writes only that import — the root `CLAUDE.md` is left untouched — since the two are separate files on disk even though they compose into one prompt at load time.

Skills — what auto-fires, what you type, and where it lives

The Skills tab is a list+detail editor over the same two folders the `claude` CLI already reads on every launch: your Skills and your Slash Commands. It does not invent a parallel system — it edits `SKILL.md` and command `.md` files directly, at whichever scope you're pointed at, and lets you browse a catalog of skills and plugins you don't have installed yet.

What you're actually looking at

Open the tab and you get one panel split into a filterable sidebar and a Monaco-backed markdown editor — the same "list+detail" shape as Hooks and MCP Servers. The sidebar groups everything into two sections, **Skills** and **Slash Commands**, sorted skills-first. A **Library** view switcher next to the scope picker swaps the whole panel for a searchable catalog instead of your installed items.

Scope	Skills live at	Commands live at
user	`~/.claude/skills/`	`~/.claude/commands/`
project	`<cwd>/.claude/skills/`	`<cwd>/.claude/commands/`

A skill is discovered by walking each scope's `skills/` root: either a direct subdir containing a `SKILL.md`, or one level deeper under a namespace directory (e.g. `~/.claude/skills/user/<name>/SKILL.md`) — the same walk a plugin's installed skills use. A command is simpler: every `.md` file directly under `commands/` is one command, named after the file.

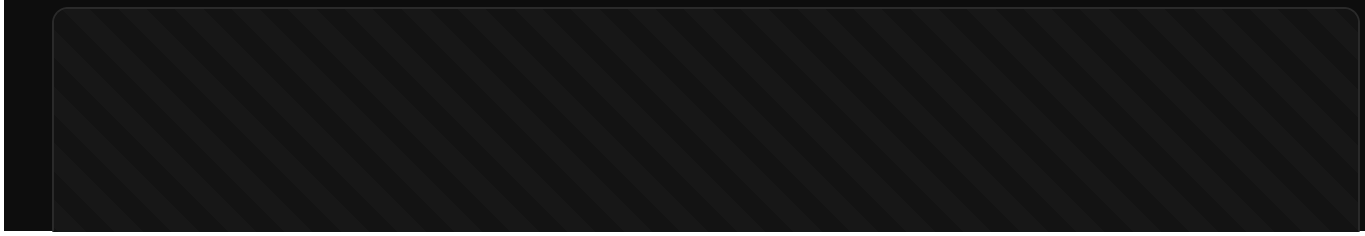


figure pending capture: Skills tab, Installed view, User scope, a skill selected in the sidebar with its SKILL.md open in the editor and the enable toggle visible

1. Skills and Slash Commands, listed separately — only Skills get the enable toggle and the remove button.
2. Scope switcher — `user` vs `project`. Defaults to whichever matches the nav face you came from, but you can override it.
3. Provenance badge — confirms exactly which scope and kind you're editing before you touch Save.

The mistake almost everyone makes: treating a Skill like a Slash Command

They sit in the same list, they're both markdown files, and it's easy to assume they work the same way. They don't:

	Skill	**Slash Command**
How it runs	Auto-invoked by the model, based on matching its own `description` frontmatter against what you're doing. You don't type anything to trigger it.	Only runs when you explicitly type `/name`. Never fires on its own.
Can you turn it off?	Yes — the Toggle in the Skills tab. It writes `disable-model-invocation: true` into the skill's own frontmatter, the actual flag the CLI reads to stop auto-invoking it. Nothing is deleted; toggling back on removes the key.	No toggle exists for commands — there's nothing to auto-invoke, so there's nothing to disable. Delete the file if you don't want it.
Can you remove it	Yes — the `×` button moves the skill's whole directory to the trash (recoverable)	Not from this tab — commands are single files;

	Skill	Slash Command
from the tab?	and re-listing it means reinstalling from the Library.	edit or delete them directly.

> If a skill keeps firing when you don't want it to, you want the **Toggle**, not the `×`. Disabling is instant and reversible; removing trashes the folder and makes you reinstall it. Confusing the two is how people end up re-downloading a skill they only meant to quiet down for a week.

The second mistake: forgetting which scope you're in

The scope switcher isn't cosmetic. Disabling or editing a skill at **user** scope changes it for every project on the machine; doing the same at **project** scope only affects the repo the active tab is pointed at. The tab defaults its scope from which nav face you arrived from — but it only re-applies that default on an actual face transition, and never once you've manually changed it — so it's worth glancing at the scope pill before you toggle anything, especially since Skills lives in the Home section of the left nav rather than under a specific project.

The Library: finding skills and plugins worth installing

Switch to **Library** and the same panel becomes a searchable catalog instead of your installed set. It has two halves reached from the same view switcher:

- **Skills** — mostly official skills from `anthropics/skills` (PDF, DOCX, XLSX, PPTX, Webapp Testing, Frontend Design, Skill Creator, and more), each with a copy-pasteable `git clone --sparse` one-liner that drops it straight into `~/.claude/skills/`. A handful of entries (the Playwright testing/visual/accessibility/ API/debug skills) are marked **local** — bundled already, nothing to install.
- **Plugins** — a much bigger catalog, and this is where this repo's own dev skillset lives: **session-manager-dev**. Unlike catalog Skills, Plugins get a real in-app **Install**

button that runs `claude plugin install <slug>` in a hidden terminal (streamed into the panel) rather than handing you a command to paste.

That distinction matters in practice: if you go looking for `/develop`, `/builder`, `/explain-to-me`, `/find-opportunity`, or `/local-project-health` in the Skills catalog, you won't find them individually — they ship together as the **session-manager-dev** plugin (installed from its own bundled marketplace, no GitHub round-trip required), and only appear as individual Skills in the Installed view *after* you install that one plugin.

The workflow that pays off

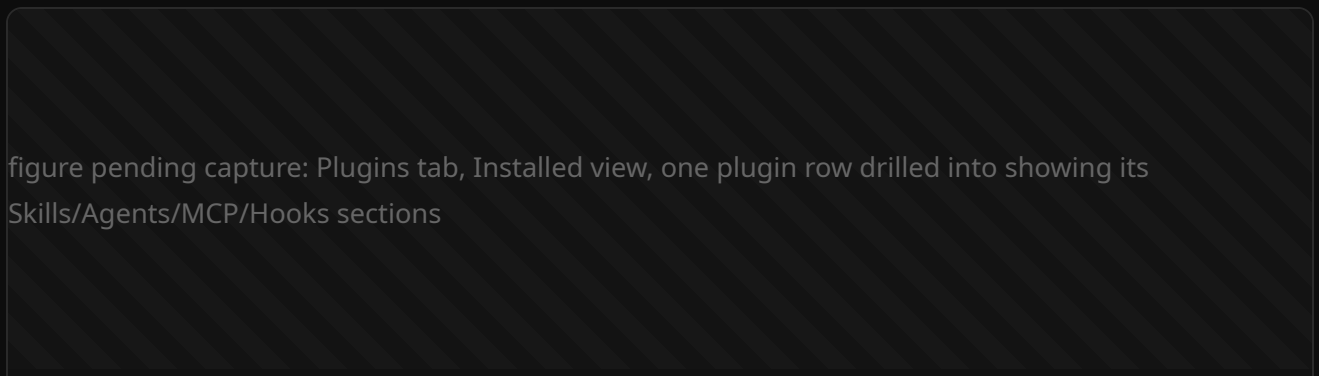
1. Before hunting for a skill by name, check the **Plugins** half of the Library first — most useful skillsets (including this project's own) ship as a plugin, not a standalone skill.
2. Install once, then switch back to **Installed** at the matching scope to confirm it landed — new skills show up with their `description` frontmatter right in the sidebar, so you can tell at a glance what will auto-fire and when.
3. If a skill is noisy but you might want it later, **toggle it off** rather than removing it — it's a one-line frontmatter edit, not a filesystem operation.
4. If you're genuinely done with it, **remove** it — the folder goes to the trash, so a change of mind a week later is still recoverable, but your Installed list stops lying to you about what's actually active.
5. Edit a `SKILL.md` directly in the tab when you want to narrow its `description` so it stops firing on the wrong prompts — that's usually a better fix than disabling it outright.

> The Skills tab never talks to a server and never fabricates what's installed — everything it shows is a live read of the same files `claude` loads on its next launch. If a skill isn't behaving the way the tab says it should, the tab is telling you the truth about the file; the mismatch is almost always scope.

Plugins — the Installed inspector and the Library catalog

Plugins is a Home-face-only Configure tab with two sub-views: **Installed**, a read-only inspector over whatever's already on this machine, and **Library**, a searchable catalog you actually install from. It is deliberately not a third place to edit Skills, Agents, or Hooks — it inspects a plugin's contents, it doesn't let you rewrite them.

View	What it's for
Installed	Reads `~/.claude/plugins/installed_plugins.json` and lists every plugin already on this machine — name, origin, version, and a drill-in showing its Skills / Agents / MCP server / Hooks / Monitors / LSP / Bin sections.
Library	A searchable catalog of installable plugins. Its Install button runs a real install, not just a copy-to-clipboard.



1. Installed / Library — the two sub-views.
2. A plugin row, drilled in — read-only, section by section.
3. A skill within that plugin, opened in a read-only body view with a list/graph toggle for how its skills reference each other.

Installed: a read-only inspector, on purpose

Clicking a row in Installed drills into the plugin's contents — the same Skills / Agents / MCP server / Hooks / Monitors / LSP / Bin breakdown any plugin can carry. This view intentionally never lets you edit any of it here: editing a plugin's own skills or agents would duplicate the Skills tab and Agent Library's UI for content that already has a canonical home there. Opening a skill from within a drilled-in plugin shows its body in a read-only markdown view, plus a list/graph toggle that visualizes how that plugin's skills reference each other.

Library: search, then Install for real

Library is a searchable catalog. Each row shows the plugin's name, its official/installed/bundle badges, a description, the exact install command as text you can copy — and an Install button that doesn't just show you that command, it runs it. Pressing Install shells out to the `claude` CLI directly: `claude plugin marketplace add <source>` first if the plugin's marketplace isn't registered yet, then `claude plugin install <slug>` (or `<slug>@<marketplace>` once one is registered) — the same commands you'd type yourself, run for you with progress streamed back into the row.

> This project's own dev skillset — the **session-manager-dev** plugin behind `/develop`, `/project-status`, `/requesting-code-review`, and the rest — is normally auto-installed the first time the app boots, not something you need to find here. Library's Install button for it exists as the fallback path: if that first-boot install is ever skipped or fails, installing it manually from here uses the identical mechanism.

Home & Project Brief — the cockpit's dashboard

Home is not a landing page you pass through on the way to work — it is the one screen that answers "what does this machine need from me right now?" across every project you've ever opened. Project Brief is its sibling on the

other side of a Terminal tab: a synthesized, editable document about the one project you're currently in. Neither one runs anything by itself; both exist to keep you from having to click through five tabs to find out what's true.

1. What Home actually shows, top to bottom

Home is assembled from existing zustand stores — nothing on this screen is a second copy of data another tab already owns. Read top to bottom, it is a fixed stack of six sections:

Section	What it is
Hero	"This machine · N projects", a live "*X* of *Y* slots busy" line from the session-slot pool, and a "*N* things need you" jump-link that scrolls straight to Needs you.
Grid	Three cards side by side: the 5-hour/weekly billing meters, known Projects, and pending scheduler jobs (Queued).
Needs you	Real signals only — a `proposed` Session awaiting Approve/Discard, a chat run stopped on `needs-input`, or a scheduler job the queue itself flagged `failed` / `needs_review`.
Active sessions	The machine session-slot pool's current holders, joined against running scheduler jobs and chat runs for context.
Recent sessions	The five most-recently-touched transcripts under `~/.claude/projects/` — the same source History reads.
Agents	A read-only reference card listing each of the five Session tags and the exact `initialPromptTemplate` text it seeds into a session — not a picker, just "what does this tag actually do".

figure pending capture: Home tab, full scroll, with at least one Needs-you row and one Active session

1 Hero — slots busy + needs-you jump link. 2 Billing / Projects / Queued grid. 3 Needs you — click a row to expand its real actions.

2. Needs you is a triage inbox, not a notification feed

Every row in this section corresponds to something that already gates on a human somewhere else in the app — Home just surfaces it in one place instead of making you go find it. Click a row to expand it inline:

Row kind	Source	Actions
Proposed Session	A Session sitting in `status: 'proposed'` — asked for, not yet started.	**Approve & start** (sends the opening prompt) or **Discard**.
Session asking a question	A chat run's most recent ticket is `needs-input` — the composer's stop-signal protocol fired mid-run.	**Open session** jumps to that Session's Terminal view.
Scheduler job failed / needs review	The queue's own status verdict on a PRD run.	**Retry now** resets the job to pending, or **View in Scheduler**.

> A job in `needs_review` is a question, not a failure — treat it the same as a chat run asking you something, not as broken work. (See the Scheduler chapter for how the root-cause report behind it gets written.)

3. The mistake people make: treating the slot slider as per-project

The "Max slots" control on Active sessions looks like it belongs to whichever project you're looking at. It doesn't. It is one number for the whole machine — the same pool the Scheduler and every Chat-view Session draw `claude -p` processes from. Defaults to 5, dragged anywhere from 0 (pauses new launches without touching what's already running) to 10. An environment override (`SM_SESSION_SLOTS`) disables the slider entirely and shows why on hover.

Drop it to 1–2 while you're actively driving a Terminal session yourself and don't want a scheduler burst competing with you for tokens; push it back up before you queue a batch of PRDs and walk away. There is no per-project equivalent — if two projects are both hammering the pool, the lever that fixes it lives on this one screen regardless of which project's tab is active.

Active sessions also explains itself when the queue is running narrower than the pool: a `memory gate` , `slots-exhausted` , `held` (behind `dependsOn`), or `paused` reason string appears next to the slot count, sourced straight from the scheduler's own last tick — it stays silent when the queue is simply idle, so an empty queue never reads as a blocked one.

4. New Session from Home is a project picker, not a form

The **New epic** button in the Hero doesn't open the New Session card directly — it opens a drawer listing your known projects (the same rows the Projects card renders) as a radio list. Confirming activates that project's tab if it's already open, or opens a dormant one, and hands you into its Sessions workspace to actually start the Session. This exists because "New Session" is scoped to one project's Terminal tab; Home's version is the cross-project entry point into that flow, not a second way to create one.

1. Click **New epic** in the Hero.
2. Pick the project from the radio list — this is real known-cwd data, not a text field.

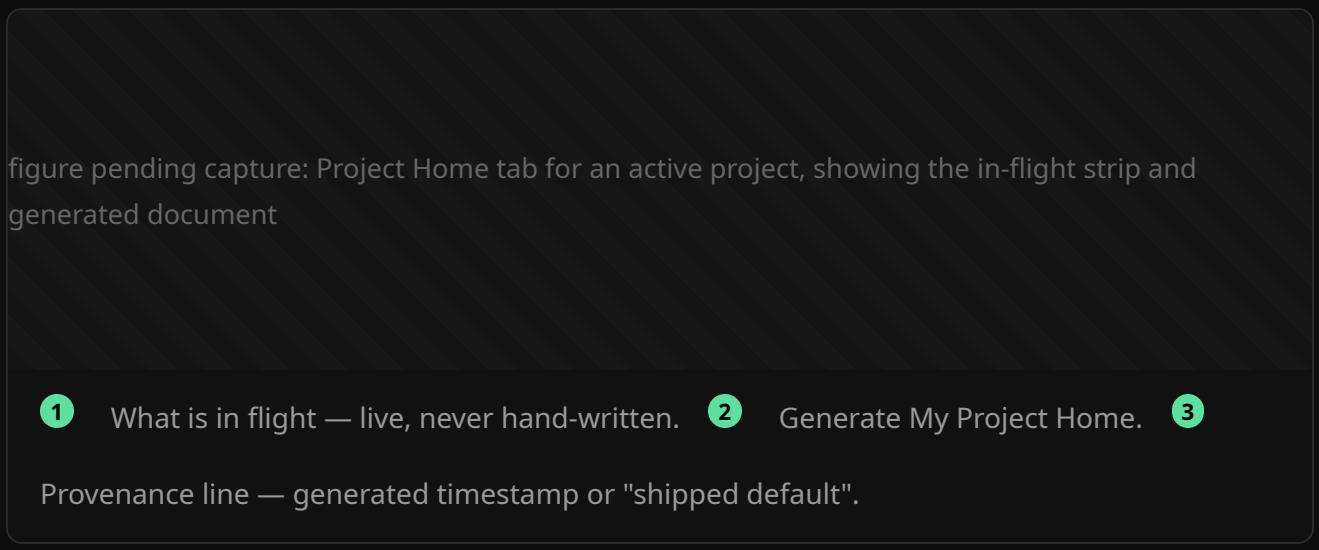
3. Confirm. The matching Terminal tab activates (or opens) and the Sessions workspace opens on top of it.
4. From there, proceed exactly as described in Getting Started: pick an Agent, pick a Mission, type your goal, submit.

5. Project Brief: a document that maintains itself, cheaply

Switch to a project's **Project Home** surface and you're looking at something different from Home: one project's own synthesized brief, not a cross-project dashboard. Above the document sits a thin live strip that changes underneath the static content:

- **What is in flight** — cards for the project's current Sessions, computed live from the Session queue every render. The card copy explicitly says "this block is never hand-written" for a reason: it can't drift out of sync with what's actually running.
- **Waiting on you** — open questions the last run couldn't resolve on its own, each with an **Answer in Session** → button that jumps straight to that Session's Terminal view. Hidden entirely when there are none.

Below that strip is the generated document itself, rendered in the same sandboxed iframe the Project Pages feature uses, with one button above it: **Generate My Project Home**.



6. What "Generate" actually does, and why it's cost-gated

Pressing Generate fires exactly one `claude -p` pass, pinned to `sonnet`, with a 3-minute hard timeout — the same spawn pattern the Memory Clusters feature uses (stdin closed, cost-gated so it only runs on your explicit click, never on a timer). It acquires a slot from the same machine-wide session-slot pool described above before it launches, so a Generate click competes with your scheduler queue and your other chat runs for the same 5 (or however many you've set) slots — if the pool is exhausted, the request simply fails with "no session slot free — try again shortly" rather than queuing invisibly.

What goes into the prompt is deliberately narrow and real: your project's `CLAUDE.md` text, every active and archived Session's tag + goal text (archived ones capped at the 200 most recent), the last 50 lines of `git log --oneline`, and a depth-2 listing of `src/`. The synthesizer is instructed to treat all of that purely as data to analyze, never as instructions to follow — it returns one JSON object and nothing else, which is then persisted to `<project>/session-manager-operations/project-brief/brief.json`.

7. The brief is a file you're meant to hand-edit

This is the detail most people miss: the brief is not a cache you regenerate and discard — it's a real, versioned document with a maintain loop built in. Certain blocks are "pinnable". Hand-edit one (zero LLM cost, no session slot involved) and it is automatically pinned, which means the next Generate pass feeds your edited text back in as an input instead of silently overwriting it with a fresh synthesis. Un-pin a block explicitly if you want the next refresh to regenerate it from scratch.

> Practical rule: generate once when a project is unfamiliar to skim its shape, then treat the brief the way you'd treat a README you maintain by hand — edit the parts that are wrong or stale, regenerate the parts you haven't touched, and don't re-run Generate reflexively just because time has passed. The provenance line under the title tells you exactly how stale the generated portion is.

8. The workflow that pays off

1. Open Home first, every session. Check Needs you before you check anything else — it's the one place a proposed Session, a stalled chat, and a failed job all surface together.
2. Glance at the slot count and the billing meters before deciding whether to queue more work or start something interactively yourself — they share the same budget.
3. Use the New Session drawer to jump into a project rather than opening a second tab for a second conversation in it; a second tab on the same cwd never activates a new project, it just re-focuses the one you already have.
4. The first time you land in an unfamiliar project's Project Home, press Generate once to get oriented, then switch to hand-editing the brief as your understanding of the project firms up — don't keep paying for full re-synthesis of blocks you already know are right.

Hooks — edit the CLI's own shell-outs, safely

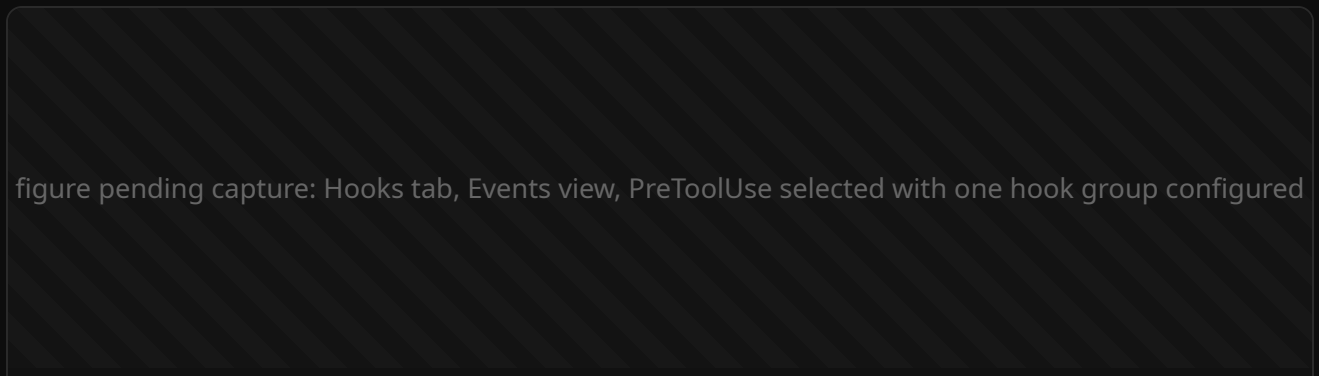
Hooks are shell commands (or HTTP calls, prompts, sub-agents, MCP tool calls) that the `claude` CLI fires on its own lifecycle events — before a tool runs, after the model stops, when a session starts. The Hooks tab is a structured editor over the exact `hooks` key in `settings.json` the CLI already reads; it doesn't add a parallel hook system, and it doesn't change what the CLI does with what you write there.

What you're actually editing

Like Settings, Hooks operates at the three file-level scopes the CLI itself resolves in order — user (`~/.claude/settings.json`), project (`<cwd>/.claude/settings.json`), and local (`<cwd>/.claude/settings.local.json`). The tab defaults to whichever

scope matches where you are (Home face → user, Project face → project) but you can switch freely with the scope switcher. Two views sit on top of the same underlying file:

View	What it shows
Effective	The merged result across all three scopes for the `hooks` key, with an override control per node. This is what actually fires — read it before assuming a hook you wrote at project scope is the one that runs, if a local override shadows it.
Events	A per-event editor for the currently selected scope's file. Pick an event on the left, add matcher groups and hook rules on the right, save.
Library (Home face only)	A static reference catalog — no cwd or scope input, identical everywhere. Not an editor.



1

Event sidebar — one row per lifecycle event, with a live count of configured hooks and a hover tooltip explaining when it fires.

2 Matcher + hook-rule editor for the selected event. 3 **test fire**

— run the command against a fake payload before you trust it in production.

The event list

The sidebar lists the current documented event set (verified against code.claude.com/docs/en/hooks): lifecycle events (`UserPromptSubmit` , `UserPromptExpansion` , `PreToolUse` , `PostToolUse` , `PostToolUseFailure` , `PostToolBatch` , `Stop` , `StopFailure` , `SessionStart` , `SessionEnd` , `Setup` , `PreCompact` , `PostCompact` , `InstructionsLoaded` , `Notification`), permission events (`PermissionRequest` , `PermissionDenied`), subagent/team events (`SubagentStart` , `SubagentStop` , `TeammateIdle`), task events (`TaskCreated` , `TaskCompleted`), state-change events (`ConfigChange` , `CwdChanged` , `FileChanged`), worktree events (`WorktreeCreate` , `WorktreeRemove`), and elicitation events (`Elicitation` , `ElicitationResult`).

> Older tool-specific event names from the 2025 era (`PreBash` , `PreEdit` , and similar) are no longer read by the CLI. If your `settings.json` still has hooks registered under one of those, the tab shows a single warning chip counting them — it does not silently keep rendering a dead UI for them. Edit `settings.json` directly to remove or rename them; the fix is on you, not the tab.

Hook rule shape

Each event holds an array of matcher groups, and each group holds an array of hook rules. A rule has a `type` — `command` , `http` , `prompt` , `agent` , or `mcp_tool` — and the field that goes with it (`command` , `url` , `prompt` , `agent` , or `mcpTool`), plus an optional `timeout` in milliseconds. Two fields are recent additions the editor already understands: `args` (an argv array for the no-shell exec form of a `command` rule) and `terminalSequence` (a terminal escape sequence, e.g. a bell, emitted alongside a notification).

The common mistake: assuming a hook works because it's saved

Saving a hook to `settings.json` only proves the JSON is well-formed — it proves nothing about whether the command runs, whether its matcher actually matches the

tool you think it does, or whether it exits cleanly. The failure mode that bites people: a hook with a typo'd path or a matcher that never matches sits there silently, doing nothing, until the day you needed it to block something and it didn't.

The fix is the **test fire** button next to every `command`-type rule. It sends the rule's resolved command a fake JSON payload on stdin and shows you the real exit code, stdout, and stderr — before you ever let the live CLI invoke it for real.

What test fire actually does

1. Pick a payload. The modal pre-fills a plausible fake event body per event type (for example `PreToolUse` defaults to `{ tool_name: "Bash", tool_input: { command: "ls" } }`) — edit it freely; it's just JSON in a textarea.
2. The tab evaluates your matcher against the payload's `tool_name` client-side first, so you see whether the rule would even fire before spending a real process launch on it.
3. Press **run**. Session Manager shows a confirmation dialog naming the exact command before executing anything — nothing fires without that click.
4. The command runs for real, with your fake payload piped to its stdin, capped at a 30-second timeout (5 s default) and a 1 MiB buffer per stream. You get back exit code, duration, stdout, and stderr.

Why this asks you to confirm every time

Test fire spawns your command with `shell: true` — deliberately, because a hook's `command` field in Claude Code is itself a shell string, so the test has to run it the same way the CLI would. That makes it the single riskiest IPC call in the app after the Watchers tab's own shell spawn. Session Manager narrows the blast radius three ways: the main window must be focused, you get a native OS confirmation dialog naming the command before it runs, and the process is hard-killed with `SIGKILL` on a timer if it wedges — `spawn`'s built-in timeout only sends `SIGTERM`, which a stuck shell can ignore.

Rule: never paste a hook command you didn't write yourself into test fire without reading it first. The confirmation dialog exists to make you look, not to be reflexively clicked through.

Workflow: authoring a hook that actually holds

1. Pick the event. Hover it in the sidebar first — every documented event has a one-line "when this fires" tooltip plus its payload shape, so you're not guessing at field names.
2. Write the matcher. Leave it empty to match every call, or scope it to a tool name/regex — an empty matcher on `PreToolUse` means your command runs before *every* tool call, which is rarely what you want.
3. Write the command, then **test fire it** against the default payload before saving anything that could block a real session.
4. Check the **Effective** view for that event afterward. If you're editing at project scope but a local-scope override already exists for the same event, the effective merge — not your unsaved draft — is what the CLI will actually run.
5. Save. Nothing fires until you do; drafts are just draft JSON in the editor.

> The Effective view is also where cross-scope conflicts hide. A hook that "isn't firing" is more often shadowed by a local override than actually broken — check there before you start debugging the command itself.

Tag Library — the mission that ships with every Session

Every Session carries exactly one tag — `feature`, `bug`, `discussion`, or one of three scripted-only tags. The tag is not a label you glance at in a queue row. Its `initialPromptTemplate` is the first framing line the agent reads, before your own goal text, and its `developEagerness` silently decides how

hard `/develop` pushes toward writing PRDs inside that Session. Tag Library is where you go to see — and, for one specific relationship, edit — what each tag actually does.

What you're looking at

Tag Library follows the same list+detail shape as Skills and Agent Library: a left rail of tags, a detail pane for whichever one is selected. Click a tag and the detail pane shows four things, straight from source — nothing here is summarized or paraphrased on the way to the screen:

- the tag's **description** (`lib/tagLibrary.ts` 's `TAG_LIBRARY`)
- its **`/develop` eagerness** — `expected-default` or `available-not-assumed`
- which mechanism actually **develops via** — most tags route through `/develop` , three don't
- the exact **opening framing** text — `agentTagDefs.ts` 's `initialPromptTemplate` , quoted verbatim, under the heading "opening framing — sent before the human's goal"

The six tags, and what each one really does:

Tag	`/develop` eagerness	Develops via	Where it's created
`feature`	`expected-default`	`/develop`	New Session card
`bug`	`expected-default`	`/develop`	New Session card
`discussion`	`available-not-assumed`	`/develop`	New Session card

Tag	`/develop` eagerness	Develops via	Where it's created
`build`	`expected-default`	`/builder` (not `/develop`)	Scripted only (EpicQueue's 'build' Session)
`project-home-builder`	`expected-default`	project-home-builder agent protocol (falls back to `/develop` only as a one-time bootstrap)	Project Home's "Generate Now" / "Regenerate" buttons
`bilko-host-publisher`	`expected-default`	bilko-host-publisher agent protocol (not `/develop`)	Scripted only

> Only three of the six show up as pills in the New Session card — `feature`, `bug`, `discussion`. The other three are a closed set too, but they're wired to a specific scripted flow (a release build, a Project Pages generation, a bilko.run publish) rather than a human picking a mission for an open-ended Session. Tag Library lists all six regardless, because all six ground a real session the same way.

figure pending capture: Tag Library tab, discussion tag selected, detail pane showing eagerness/develops-via/opening-framing

1. The tag rail — six tags, agent count per tag.
2. `/develop` eagerness and develops-via, read straight from code.
3. The opening framing text — exactly what the agent reads first.

What's actually editable here — and what isn't

It's easy to assume "Library" means the whole page is a CRUD editor, the way Agent Library and Skills are. It isn't, and the distinction matters:

- **The agent ↔ tag relationship is editable, right here.** Each Agent Library persona carries its own `tags` frontmatter field. The "agents carrying this tag" row in the detail pane lets you assign or remove a persona from a tag with one click — it writes through the same `savePersona` call Agent Library's own tag-assignment chips use, so there's exactly one source of truth for that relationship no matter which page you edit it from.
- **The tag taxonomy itself is not editable from the UI.** The six tags, their descriptions, and their `initialPromptTemplate` text are a closed TypeScript union (`EpicTag` in `lib/tagLibrary.ts`) threaded through `composeEpicIntake` , `ticketDisplayStatus` , and the Scheduler's group ordering. Changing what a tag says, or adding a seventh tag, is a source edit to `lib/tagLibrary.ts` and `lib/agentTagDefs.ts` , not a form on this page.

In practice this means: if you want a specific agent to always pick up `bug` Sessions, do it here. If you want `discussion` itself to phrase its framing differently, that's a code change to `agentTagDefs.ts` 's `initialPromptTemplate` for that tag — Tag Library will show you the result the moment it lands, but it won't let you type it in.

The mistake: treating the tag as decoration

The tag's `initialPromptTemplate` is not a description shown somewhere in a sidebar — it is composed directly into the Session's opening prompt, before the human's own goal text. `lib/epicIntake.ts` 's `composeEpicIntake` builds every Session's first message in a fixed order — Actor, then any Context Injections, then Input, then **Mission** (the tag's `initialPromptTemplate`), then the human's own goal, then references. Every single Session tagged `discussion` , for example, opens with:

```
You are in an open-ended discussion. The goal is a decision or shared unde
not a code change — do not start editing files unless the human explicitly
```

```
after the discussion lands somewhere.
```

That's not cosmetic. It's the sentence standing between "the agent explores the codebase and tells you what it thinks" and "the agent starts refactoring files because the goal text sounded actionable." A vague or missing framing line does the same damage a vague PRD does in the Scheduler — it pushes the interpretation burden onto every future Session that inherits the tag, instead of settling it once in the template.

The second half of the same mistake is ignoring `developEagerness`. It isn't just informational text in the detail pane — the `session-manager-dev:develop` skill reads it to decide how hard to push toward PRD decomposition inside that Session. `feature` / `bug` (and `build` / `project-home-builder` / `bilko-host-publisher`) treat decomposition as the expected next step; `discussion` keeps `/develop` available but never assumed until whether-to- build is actually settled. Picking `feature` for what's really an open-ended discussion gets you an agent that's eager to start writing PRDs before you've decided anything is worth building.

The workflow that pays off

Because `composeEpicIntake` pulls the Mission section fresh from `agentTagDefs.ts` every time a Session is created, a template edit is retroactive in effect for every *future* Session without touching a single Session record — you author the tag once and every Session with that mission inherits it from that point on. That makes tightening a tag's template one of the highest-leverage edits in the whole app: fix a template's ambiguity once, and you've fixed the framing for every discussion/bug/feature Session you'll ever open with it, not just the one in front of you.

1. Open Tag Library and read the tag's **opening framing** block exactly as an agent would — as the first thing it's told, with no context yet about your actual goal.
2. If it's vague enough that two different agents could read it two different ways, that's the bug. Tighten `initialPromptTemplate` in `agentTagDefs.ts` — state the one behavior you always want for that mission (reproduce before fixing, no

file edits until a decision lands, treat the goal as the full objective) rather than a generic description of the tag's name.

3. Check `developEagerness` against what you actually want that mission to do by default. If a tag's Sessions keep spinning up premature PRDs, or conversely never seem to reach `/develop` when they should, this field — not the goal text you type per-Session — is the lever.
4. Use the agent↔tag assignment in the detail pane to make sure the personas you actually run under a given mission are the ones associated with it — it's the one relationship this page will let you change without leaving it.

> The taxonomy stays deliberately small and closed. Before reaching for a seventh tag, check whether what you actually want is a different *Agent* (the "who") rather than a new *Mission* (the "what") — Agent Library is the other half of this axis, and it's usually the cheaper edit.

Host on Bilko.run — publishing this project's page, gated for real

Host on Bilko.run is a Project-face-only Configure tab that takes this project's generated Project Page and puts it live on bilko.run, at `bilko.run/projects/<slug>/`. It's a thin cockpit over Bilko's own publish pipeline — this tab never writes to that site directly; every real publish goes through a gated MCP tool chain and a dedicated Epic that runs the actual checks.

> The Session Manager **app is free** and stays free — hosting a project on bilko.run from this tab is an ordinary app feature, not something gated behind a purchase. The Field Manual you're reading is the only thing Session Manager sells; nothing in this tab, or anywhere else in the app, checks a license or entitlement before letting you publish.

Two stages: a free local build, then a gated publish

Stage	What runs	Trigger
Stage A — Prepare Bundle	Deterministic, no agent involved: rebuilds a local static bundle from your current hosted-documents list into `session-manager-operations/bilko-host/dist/`.	**Prepare Bundle** button. Free, idempotent, safe to re-run any time the document list changes.
Stage B — Publish	Starts a Session tagged `bilko-host-publisher`, which calls Bilko's own `bilko-host` MCP tools to register the slug (first publish only) and push the bundle through five gates — manifest schema, size budget, a golden-path check, an accessibility scan, and a dependency audit.	**Publish** button — disabled until a bundle exists. If a publish is already in flight, it turns into **View publish in progress** instead of starting a second one.

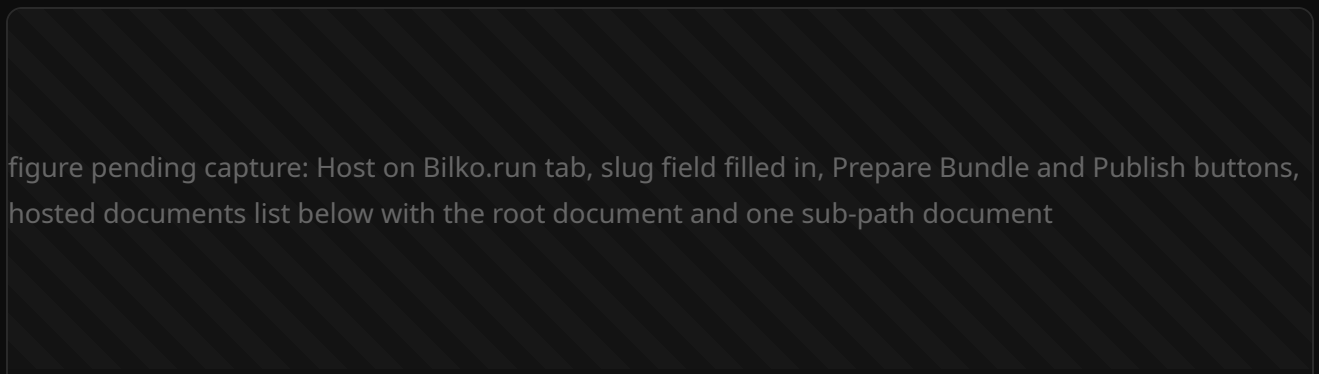


figure pending capture: Host on Bilko.run tab, slug field filled in, Prepare Bundle and Publish buttons, hosted documents list below with the root document and one sub-path document

1 Slug field — `bilko.run/projects/<slug>/`. 2

Prepare Bundle, then Publish — the two-stage pipeline, one button each. 3

Hosted documents — the root page plus any sub-path pages you've added, each removable.

One project, one Bilko listing, many pages under it

A Bilko listing isn't limited to a single page. The root document (published at the slug's own URL) defaults to this project's Marketing Project Page, but you can add any number of additional documents at their own sub-paths — another Project Page lens

(Home, Feature, or Architecture) hosted as its own page, or an arbitrary local HTML file from this project. Each one lands at `bilko.run/projects/<slug>/<subpath>/`.

> Removing a document from the list here takes effect immediately in your local bundle, but stays live on bilko.run until your *next* Publish — Bilko's publish tool replaces the entire listing wholesale each time, which is also what guarantees a removed document actually disappears rather than lingering as an orphaned file.

Reading a publish failure

If a gate rejects the bundle, the status card shows **Last publish failed:** followed by that gate's own error message — never a generic "something went wrong." The publishing Session reports exactly which of the five gates failed and does not attempt to bypass it on its own initiative; overriding a failing gate is a decision only a human makes, from that Session's own conversation.

History — where your Claude time and money went

History is an analytics dashboard, full stop. It answers "how many tokens, how much money, which project, which model, which tool" over a date range. It does not list your past Claude sessions, and it cannot reopen one — that used to be true, and the app was deliberately cut in half to stop it being true.

The mistake almost everyone makes once

The instinct is to open History looking for a session you had last week — "what did I ask Claude to do in that project on Tuesday?" — expecting a searchable log with a resume button. History does not do that, and it used to: a session-browsing "Log" view with a per-row **resume** button lived inside this tab until it was surgically split out into its own component and left unrendered, pending a future relocation onto the Projects or Terminal surface where session browsing actually belongs. The History tab you see

today renders only the analytics dashboard — the browse/resume UI was ruled out of its charter, not merely hidden behind a toggle.

> If you're hunting for an old conversation to resume, this is the wrong tab. Nothing currently in the app exposes that browse-and-resume flow — it's parked, not shipped.

What it's actually a browser over

History reads from a durable rollup store built and maintained on disk — it never re-scans your raw transcript files to render. That's what lets the dashboard open instantly no matter how many sessions you've accumulated. Every row it aggregates carries: prompt count, input/output/cache tokens, tool-call count, session count, error count, active minutes, and an estimated dollar cost per model, bucketed per project per day.

Cost is an *estimate*, not a billed figure — computed from a per-million-token pricing table keyed by model family (Opus, Sonnet, Haiku), matched against the raw model id by a case-insensitive substring check. A model id that doesn't match any known family still gets priced (falling back to the Sonnet rate) but is flagged internally as an estimate rather than an exact figure, since cache-read tokens are billed far below a fresh input token and the fallback can't know the real family's ratio.

Seven measures, one dial

Every panel in the dashboard reads off the same measure — you change it once, in the control bar, and every chart re-renders against it:

Measure	What it counts
In tokens	Input + cache-read + cache-creation tokens — everything submitted to the model.
Out tokens	Output tokens only.

Measure	What it counts
Total tokens	In + Out.
Prompts	Number of prompts sent.
Sessions	Number of distinct sessions.
Time	Active minutes.
Spend	Estimated dollar cost. The default measure on first open.

Range is a separate dial next to it: 30, 60, 90 days, or "All time". Both choices persist to local storage, so History reopens the way you left it.

One entry point: the Home face

History lives on the Home face only — there is no per-project nav row for it any more. That's deliberate, not a missing feature: the dashboard already folds every project together on disk (the same rollup store underlies every view), so a per-project copy of History was a second door onto the identical cross-project screen. Reach it from the Home sidebar or the command palette; both always land you on the full, every-project dashboard.

Once you're there, the **Project facet** chip row does the narrowing a per-project tab used to do — toggle individual projects, isolate one, or show all. It's a filter you set inside the one dashboard, not a different scope you navigate to.

Reading the panels top to bottom

1. **Headline** — the current measure's total for the range, with a delta against the equivalent prior period and a small trend series.

2. **Budget strip** — month-to-date spend against a cap you set (defaults to $50, persisted locally), projected out to end-of-month from the days elapsed so far. This one is deliberately unfaceted — a budget is a real-money figure for the whole account, not scoped to whatever projects you're currently filtering.
3. **Project facet** (Home face only) — chip row to toggle/isolate projects before the rest of the panels compute.
4. **Stacked trend** — the measure over time, stacked by project. Click a project's band to drill into it.
5. **Ranking** — every project in range, sorted by the current measure, with its share of the total and a first-half-vs-second-half trend arrow.
6. **Project drill-in** — appears once you select a project from the trend or the ranking table. Breaks that one project's totals down by model (sorted by cost) and by tool call count.
7. **Model mix, Concentration, Cache, Rhythm** — four smaller cards: spend/tokens by model family, how much of total spend the top 3 projects account for, your prompt-cache hit rate, and which day of the week you actually work on.

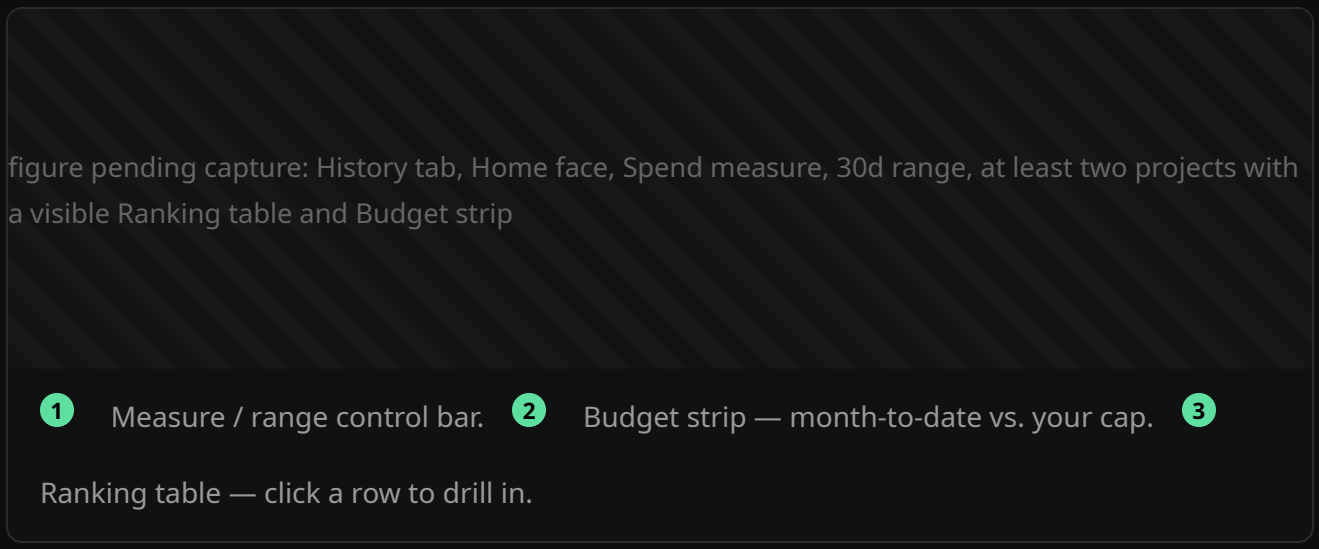
The workflow that pays off: catching a runaway project before the bill does

The dashboard exists to answer one operator question fast: *is something quietly burning tokens I didn't budget for?* The path that actually catches it:

1. Switch to the Home face and set the measure to **Spend**.
2. Check the **Budget strip** first — if the projected month-end figure is already over your cap, something changed recently.
3. Look at **Concentration**. If the top-3 share is unusually high, one or two projects are doing the damage, not a broad increase across everything.
4. Open the **Ranking** table and click into the top row's **drill-in**. The by-model breakdown will usually tell the story on its own — a Session accidentally pinned to

Opus instead of Sonnet shows up here as a cost-per-token outlier long before it shows up on an invoice.

5. If the cache-hit percentage in the **Cache** card is low for that project, that's often the actual lever — a session re-sending large context on every turn instead of hitting the prompt cache costs real money at input-token rates.
6. Fix it at the source: change the offending Agent persona's `model` field, or split an overgrown Session into smaller ones. History only tells you where the money went — it has no control to throttle or gate spend itself.



When a project silently goes missing from the totals

The underlying store keys every day's data by the encoded `~/.claude/projects/<dir>` transcript folder name — a transcript-store identity, not a project identity, since the CLI mints a folder for every path it was ever launched from, including throwaway `/tmp` directories and test fixtures. History resolves each folder back to a real working directory and merges folders that resolve to the same project (their token counts, tool breakdowns, and per-model figures all combine into one row) before anything renders. A folder whose cwd can't be resolved at all is excluded from every chart's project axis — but never silently: its totals are summed and shown in a dedicated exclusion note beneath the headline ("Excluded: N transcript folders with no resolvable working directory · … sessions · … prompts · $…"), so an unrecognized dollar amount is always stated rather than dropped. If you ever see that note, it means

History deliberately declined to guess which project the folder belonged to — it will never fold unrecognized activity into a project you didn't actually run it from.

Exporting the raw numbers

The **Export CSV** button in the control bar writes exactly what's on screen — the currently filtered range, faceted to whichever projects are shown — as a per-day CSV with per-model token and cost breakdowns, named after the range's start and end date. Useful for feeding a spreadsheet or an expense report; there is no other export path out of History.

Memory Clusters — what an agent has actually been remembering

Memory Clusters is a read-mostly report over one project's saved memory notes — it groups them into named themes so you can see, at a glance, what Claude has decided is worth remembering about your codebase. It replaced the old Knowledge Graph tab. It is not live, it is not this conversation's context, and it costs a real `claude -p` call every time you ask it to regroup.

Where the memories actually live

The Memory tab (Configure nav) has two scopes — Workspace and Subagent — plus, inside Workspace, three views: **Editor**, **Natural**, and **Clusters**. All three read the same on-disk store: `~/.claude/projects/<encodedCwd>/memory/*.md` — Claude's own native auto-memory location for that project, not something Session Manager invented. Editor is a plain list+detail CRUD over those files; Clusters is a report generated *from* them. Nothing you do in Clusters writes back to a memory file — it only reads and summarizes.

Each memory is a markdown file with optional frontmatter (`type` , `description`) and a body that may contain `[[wikilink]]` -style references to other memory slugs.

Claude writes these during ordinary sessions when you ask it to remember something; you can also create one by hand from the Editor view's **+ New memory** button.

The one thing people get wrong

Memory Clusters does **not** update as a session runs, and it is **not** a view into the current conversation. Two separate confusions come from that one fact:

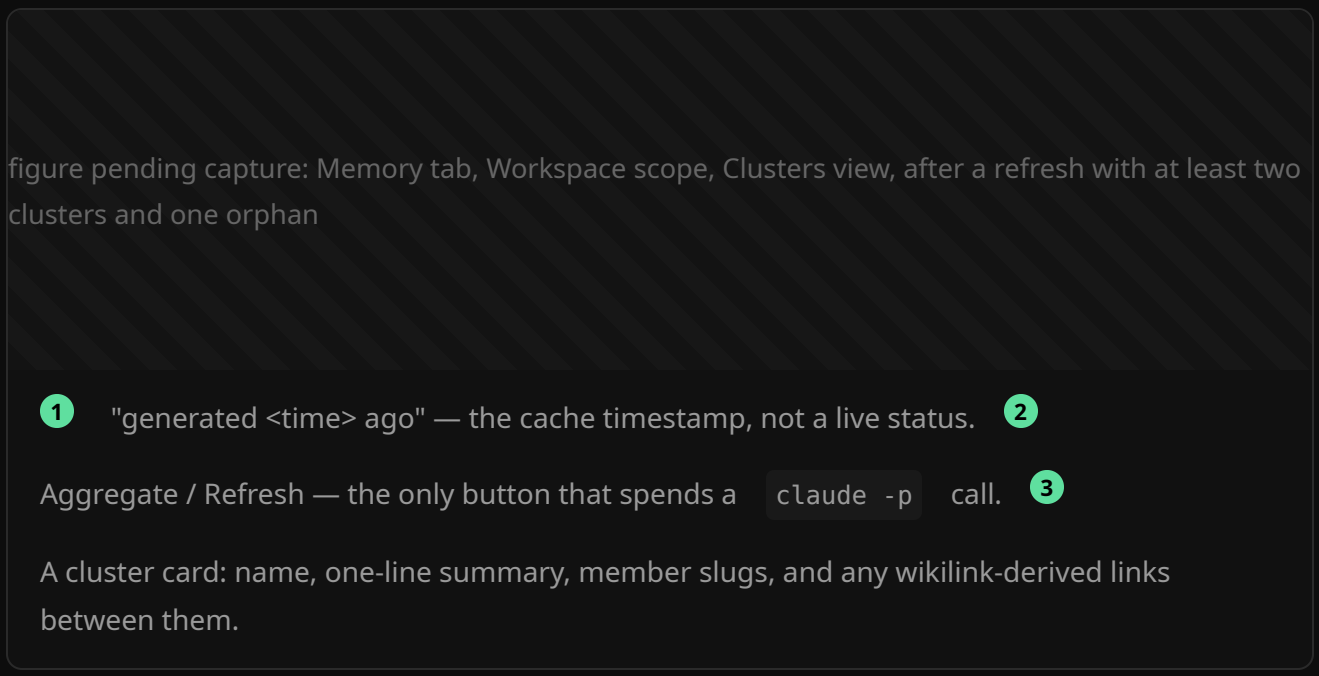
Expectation	Reality
"It'll update live while my agent works."	It's cache-only until you press **Aggregate**/**Refresh**. Opening the Clusters view (or switching workspaces) only re-reads the last cached result — it never spends a token on its own.
"This shows what we've been talking about in this Session."	It shows the persisted memory *files* for the project — durable notes an agent chose to write down — not the transcript of any one session. A long conversation that produced zero `memory` writes shows up as nothing here.

What actually happens when you press Refresh

The button is the cost gate, by design. Pressing it sends every memory file for the active workspace — slug, type, description, body — into a single `claude -p` call pinned to the `sonnet` model, with a system prompt that explicitly tells the model to treat memory bodies as inert data to analyze, never as instructions to follow (a memory file is user-writable text; without that framing, a maliciously or accidentally instruction-shaped memory body could otherwise hijack the clustering pass). The model returns 2–8 named clusters, a one-line summary per cluster, member slugs, any `[[wikilink]]`-derived links between them, and a list of orphans — memories that fit no cluster.

1. You open the **Clusters** view. Session Manager reads the cached result only — no spend.

2. You press **Aggregate** (first run) or **Refresh** (subsequent runs). This is the only action that fires the LLM pass.
3. Every workspace memory file is read and sent as data, wrapped in an explicit do-not-follow-instructions framing.
4. The result is parsed defensively — any cluster member, link endpoint, or orphan that isn't a real memory slug on disk is dropped, so the report can never reference a memory that doesn't exist.
5. The result is cached to `~/.claude/session-manager/memory-clusters/<workspace>.json` and rendered as cluster cards plus an "Unclustered" strip for orphans.
6. Clicking any slug — in a cluster or in Unclustered — jumps you to that memory in the Editor view.



If the tab looks empty

An empty Clusters view almost always means one of two things, in order of likelihood:

- **You haven't pressed Aggregate yet.** A fresh workspace with no cached result shows "not yet aggregated" and an empty state — this is expected, not a bug.

- **This project genuinely has no memory files yet.** If Claude hasn't been asked to remember anything for this cwd, `~/.claude/projects/<encodedCwd>/memory/` is empty or doesn't exist, and there is nothing to cluster. Check the Editor view's entry count in the toolbar — that's the ground truth for how many memory files actually exist.

> **A slow prior workspace can't clobber a fast current one**
>
> Switching workspaces (i.e. switching the active project tab) while a Refresh is still in-flight used to risk the old workspace's slower response landing after the new workspace's faster one, showing the wrong project's clusters. Each load request is tagged with a request id, and a response is only applied if its id still matches the latest request — a stale response is silently discarded rather than painted over the current workspace.

The workflow that pays off

Treat Clusters as a periodic audit, not a live dashboard: refresh it every so often — after a long Session, or before starting a new one on a project you haven't touched in a while — and read what an agent decided was worth writing down. It's a fast way to catch two things: memories that drifted into duplicating something already in a CLAUDE.md (a candidate for `/memory-sanitation`), and orphaned notes that never got tied into anything, which often means they were written once and never referenced again.

> Clusters is read-only reporting layered on top of the Editor view's real CRUD. If a cluster's grouping looks wrong, the fix is editing or deleting the underlying memory file in Editor, then pressing Refresh again — not fighting the clustering prompt.

Usage — reading the 5-hour window

before it reads you

Every `claude` invocation this app makes — your interactive Terminal, a Chat run, a Scheduler job — draws against the same rolling billing windows Anthropic tracks for your account. Session Manager surfaces those windows in three places and gives you one lever to react to them: the machine-wide session slot pool. This chapter is about noticing the ceiling before you hit it, not after.

Where usage is shown

Surface	What it shows	Where to find it
Home — usage meters	Session (5-hour), Weekly (all models / Opus / Sonnet / OAuth apps), and extra pay-as-you-go usage if enabled — each with a percentage, a progress bar, and a reset countdown.	Home tab, billing card
Almanac footer — session & weekly pills	Two numbers side by side: `session N%` (the 5-hour window) and `weekly N%` (the 7-day, all-models window). Both click through to Home.	Bottom status strip, every tab
Almanac footer — scheduler-paused pill	Appears only while the Scheduler is paused; states the reason (`rate_limit`, `auth`, `network`, …). Click it to jump to Scheduler.	Bottom status strip, conditional
Home — session slot control	The machine-wide concurrency pool: how many `claude -p` processes may run at once, and how many are busy right now.	Home tab, Active sessions card

figure pending capture: Home tab, billing card, Session meter above the Weekly rows

1. Session — the 5-hour rolling window, rendered larger than the rest.
2. Weekly rows — all models, Opus, Sonnet, OAuth apps, each independent.
3. Reset countdown — relative time plus the absolute wall-clock time in Pacific.

What the numbers mean

Every meter is a `{ utilization, resets_at }` pair read from the same endpoint the CLI's own `/usage` command hits. Utilization is a percentage that can exceed 100 — the UI keeps the bar clamped at full width but still prints the real number and marks the row **over limit** once it crosses 100.

Threshold	Color	Meaning
< 70%	sage (calm)	Plenty of headroom.
70–89%	honey (caution)	Getting close — a good moment to check what's queued.
≥ 90%	accent (alert)	About to be rate-limited, or already over.

Reset times are shown two ways: a relative countdown (`2h 14m`, or `4d 3h` once it's more than a day out) and an absolute wall-clock time in Pacific (`3:00 PM PT`, or `Tue 3:00 PM PT` for a multi-day reset) — so you don't have to do timezone math to know when a paused Scheduler will pick back up.

Where the numbers come from

The main process polls `GET https://api.anthropic.com/api/oauth/usage` — the same endpoint the CLI's own `/usage` slash command reads — authenticated with the

OAuth bearer token from `~/.claude/.credentials.json` (written by `claude login`; no separate API key needed). One renderer-side poller ticks every 60 seconds and every tab reads from the same cached result, rather than each surface hitting the endpoint on its own timer. A successful fetch is cached for 30 seconds so a burst of reads doesn't re-hit the network; a transient failure backs off to 5s then 30s, and an auth failure backs off to 30s.

If your account authenticates through an enterprise gateway — Bedrock, Vertex, a raw API key, a custom `ANTHROPIC_BASE_URL`, or an `apiKeyHelper` script — this meter simply doesn't exist for you, and Session Manager detects that rather than treating a 404 as a real outage: the Scheduler does not pause waiting on a signal that will never arrive.

The session slot pool

Every `claude -p` process this app launches — a Scheduler job or a headless Chat run — first has to acquire a slot from one machine-wide pool. It exists because Scheduler and Chat used to enforce their own private caps (3 and 2) that could add up past what the machine could actually hold; the 2026-06-10 OOM was exactly that. Now there's one number, and one place to change it.

Setting	Value
Default	5 concurrent `claude -p` processes
Adjustable range	0–10, from the slider on Home's Active sessions card
0	Pauses new launches without touching processes already running
`SM_SESSION_SLOTS`	Env var override for scripted/CI use — pins the pool size and disables the slider (clamped to the same 0–10 range)

The slot pool and the 5-hour billing window are two different ceilings and it's worth keeping them straight: the slot pool caps how many processes run *at once* on this

machine; the billing window caps how much you can send *in a rolling five hours*, machine-independent. Turning the slot pool down does not buy you more billing headroom — it only slows how fast you spend the headroom you have.

The common mistake

The failure mode isn't that the app hides rate-limiting — it's that nothing forces you to look. The Scheduler auto-pauses cleanly on a rate limit and auto-resumes at the next window reset, which is exactly the point: you're not supposed to have to babysit it. But that same quiet handling means a queue can sit paused for hours with nothing on screen demanding attention unless you're on the Scheduler tab or you happen to glance at the footer. The scheduler-paused pill only appears in the footer while paused, and it's easy to miss if you're heads-down in a Terminal tab that isn't the active one.

The workflow that pays off

1. Treat the footer's 5h pill as a peripheral gauge, not something you check on a timer — it's visible from every tab, so glancing at it costs nothing.
2. When it crosses into honey (70%+), open Home and look at which Weekly row is also climbing — a Session window resets in hours, but a Weekly window doesn't.
3. If the scheduler-paused pill appears, click straight through to Scheduler rather than wondering why nothing's moving — the reason is stated on the pill itself.
4. Size the session slot pool to your plan, not to your machine's core count. A higher plan tier with more 5-hour headroom benefits from more slots running in parallel; a lower tier just burns through the window faster for no extra throughput once jobs start queuing behind the rate limit anyway.
5. Leave `SM_SESSION_SLOTS` for scripted/CI contexts only — setting it also disables the Home slider, which is easy to forget you did.

> The 5-hour window is the same meter across Terminal, Chat, and every Scheduler job — there's no per-surface budget. Watching it in one place covers all three.

Voice Input — dictate into any session, offline

The mic button turns your microphone into a keyboard for the active Terminal or Chat view. Speech never leaves the machine: recognition runs in a Web Worker, in-process, and the model downloads once and is cached. It is not a convenience wrapper around a cloud API — it is a real local speech pipeline, with its own state machine, its own auto-submit timer, and its own failure modes worth understanding before you rely on it mid-session.

1. What's actually running

The app's own mic-button tooltip calls this "local Whisper" — that name is legacy. Read the worker source (`whisperWorker.ts`) and the real model is **Moonshine Base** (`onnx-community/moonshine-base-ONNX`), loaded through Transformers.js and run on the CPU via `onnxruntime-web` (`dtype: 'fp32'` , `device: 'wasm'`). Moonshine was chosen over Whisper deliberately: Whisper zero-pads every input to a fixed 30-second window before encoding, so even a two-word command pays a fixed latency floor; Moonshine's encoder cost scales with the actual audio length, so a two-second utterance transcribes in roughly 150 ms once the model is warm.

Speech detection is a second, separate model: **Silero VAD v5** via `@ricky0123/vad-web` , running on every ~32 ms audio frame through an `AudioWorklet` . VAD decides when you started and stopped talking; Moonshine only ever sees the speech segment VAD hands it — no fixed-length chunking, no silence sent to the model. Both the VAD's ONNX weights and the WASM runtime are self-hosted under the app's own `/vad/` assets rather than fetched from a CDN, because a CDN fetch breaks under the packaged app's `file://` origin.

Layer	What it is	Runs where
Endpointing	Silero VAD v5, `@ricky0123/vad-web`	AudioWorklet, per 32ms frame
Transcription	Moonshine Base (`onnx-community/moonshine-base-ONNX`), via `@huggingface/transformers` + `onnxruntime-web`	Shared Web Worker (`whisperWorker.ts`)
Language	English only — the worker's own source comment flags non-English audio as hallucinated English	—

> First activation downloads the model (tens of MB) and shows a percentage in the mic button's tooltip and the Voice modal's progress bar. After that it's cached and loads from disk — expect the first click of a session to take longer than every click after it.

2. Turning it on

1. Click the mic icon wherever it appears (Terminal toolbar, Chat composer) — or run **Voice — toggle mic** from the command palette (Cmd/Ctrl-K).
2. The very first click on a fresh install opens the first-run mic-check wizard instead of recording — it walks permission → device selection → a 5-second test recording → playback → a real transcription check, so you find out your mic works before you're mid-task depending on it. "Skip & record now" closes the wizard and starts recording immediately.
3. Grant the OS microphone permission when prompted. A denial parks the button in a permission-denied state — the tooltip tells you to fix it in OS settings, not in the app.
4. Speak. Once VAD confirms real speech (not a keyboard click or a chair creak), the mic ring goes solid and a live "Listening…" line shows in the Voice modal.

You can also set a global or in-window hotkey (default `Cmd+Option+V` on macOS, `Ctrl+Shift+Space` on Linux/Windows — both deliberately chosen as combinations that ship unbound in stock GNOME/KDE/macOS) with two modes: **Hold to talk** (push-to-talk — release the key to stop) or **Tap on/off** (press once to start, again to stop). Configure both from the Voice / Microphone modal.

figure pending capture: Voice / Microphone modal, open, mid-recording with the level meter active

1

Live activity line — shows the committed transcript or "Listening..." while VAD has an open speech segment.

2 Auto-submit countdown bar — the thin strip under the activity line. 3

Hotkey mode toggle — Hold to talk vs. Tap on/off, plus the mic device picker.

3. The mistake almost everyone makes: talking over the auto-submit timer

By default, every time VAD commits a final transcript, Session Manager writes the text into the active PTY (or the Chat composer, in Chat view) and arms an **8-second** countdown (`submitDelayMs` , adjustable between 500 ms and 13 s). When it expires, the app sends Enter for you — no second confirmation. That's the whole point: dictate a sentence, stop talking, and it submits itself a few seconds later without you touching the keyboard.

The mistake is assuming that countdown is dumb. It isn't — it's actively watching you. Every VAD frame's speech probability is checked against the countdown in real time (`SUBMIT_PRECEDENCE_PROB = 0.3` , well below VAD's own 0.5 segment-commit threshold), and an RMS amplitude fallback (`SUBMIT_PRECEDENCE_RMS = 0.04`) covers

the case where the VAD frame pump stalls. The instant either trips, the countdown is cancelled and the mic stays open — so saying "wait, actually—" mid-countdown correctly holds your Enter key hostage rather than firing it under your correction. What actually bites people is the opposite case: dictating a full thought, going quiet to think about the *next* sentence, and having Enter fire on the first one before they resume talking. The countdown bar under the mic button (a thin strip that turns amber in the last 500 ms) is the tell — watch it, or click it to cancel outright.

Two other timers run alongside the submit countdown and are easy to conflate with it:

Timer	Fires after	What it does
Auto-submit	8 s (default) of silence *after a transcript typed*	Sends Enter, then keeps the mic open — this is *continuous listening*, not one-shot. The session doesn't close after a turn.
Idle auto-stop	40 s of silence with no speech at all	Closes the mic outright — the real "you walked away" backstop.
Model-load watchdog	90 s with no download/warmup progress	Flips the button to an error state you can click to retry, instead of hanging forever on a stalled fetch.

> Auto-submit doesn't end your dictation session — only the idle-stop timer or an explicit stop (button click, hotkey, or voice command) does. Say your first sentence, pause, think, say your second: both land as separate submitted turns without you touching the mic control between them.

4. Turn continuous dictation off, or don't submit at all

Two dials worth knowing about beyond the button and the hotkey:

- **Cancel a single countdown** — click the thin progress bar, or run **Voice — cancel auto-submit** from the command palette. The typed text stays in the PTY; you press Enter yourself when ready.

- **Turn auto-submit off entirely** — the store's `autoSubmit` flag (default `true`) makes dictation type-only: text lands, no Enter is ever sent, no countdown ever arms. Use this if you dictate multi-line prompts you want to review before sending.

5. Where transcripts go, per view

Voice input targets whichever tab is active when you start recording, and the destination depends on that tab's kind:

Active view	Final transcript goes to
Terminal (live PTY)	Written straight into the PTY via `pty.write`, exactly like typed keystrokes — including voice commands.
Chat / dormant tab	Buffered as the tab's pending voice text and handed to the Chat composer's own `send()` path when the auto-submit countdown fires — there's no PTY to write into on a headless `claude -p` tab.

A handful of spoken phrases are matched as **voice commands** before they're treated as literal text (Terminal view only — dormant/Chat tabs always treat a transcript as literal, since terminal control sequences have no meaning in a headless run). A recognized command sends its control sequence directly and skips the submit countdown outright, since the command *is* the submission.

6. What "offline" actually means here

Nothing about a spoken phrase leaves your machine at any point in this pipeline — VAD, encoder, and decoder all run in-process in a renderer Web Worker. The privacy corollary is enforced, not just promised: whenever `isRecording` is true (or an external, non-terminal capture like the Document Experience's own dictation is active), a fixed red banner — **"Recording — speech is being captured locally"** — is pinned to the very top of the window, above every dialog, toast, and menu in the app's z-order. There's no code path that starts a recording without that banner in front of you.

figure pending capture: app window with the top-of-window red recording banner visible during an active mic session

1

The privacy banner — always the topmost layer in the app while recording, on every platform, with no way to dismiss it short of stopping the mic.

7. If the mic button won't cooperate

Button state / tooltip	Meaning	Fix
Grey, spinning, "loading… N percent"	Model still downloading or warming up.	Wait. If it sits past ~90s with no percent movement, it self-flips to an error you can click to retry.
Amber, "permission denied"	OS denied microphone access.	Fix it in OS privacy settings, then click the button again — this isn't an app-level toggle.
Red, "model error" / "load timed out"	Download or warmup failed, or the 90s watchdog tripped.	Click the button — clicking an error/timeout state retries the load instead of trying to record.
Grey, disabled, "not supported"	The runtime lacks `Worker`, `getUserMedia`, or `AudioWorkletNode`.	Not fixable in-app — this build/environment can't run the pipeline at all.

> The workflow that actually pays off: leave auto-submit on, trust the countdown bar as your only feedback loop, and use "Hold to talk" if you'd rather a released

> key be the unambiguous stop signal than a silence timer. Once that's second nature, voice stops being a novelty input and becomes a genuinely faster way to drive a Terminal or Chat session than typing.

Plans & Tasks — reading what the agent already told you

Session Manager does not run a second task manager next to Claude Code. Every checklist and every plan you see in the app is read straight out of the same transcript JSONL the CLI already writes to `~/.claude/projects/<encoded-cwd>/<sessionUuid>.jsonl`. There is no separate place to "add a task" — if it isn't in the transcript, it doesn't exist here either.

Where this data actually comes from

Two Claude Code tool calls carry the whole feature. `src/main/lib/classifyTranscriptLine.cjs` watches every `tool_use` block in the tail and reclassifies exactly two of them:

Tool call the CLI made	Transcript event kind	What it means
`TodoWrite`	`todo_write`	The agent replaced its whole to-do list. Session Manager stores the *latest* list only — each new `TodoWrite` call overwrites the previous one, it doesn't append.
`ExitPlanMode` / `EnterPlanMode`	`plan`	The agent proposed or revised a plan while in plan mode. The plan text itself comes from the tool call's `plan` input field.

Both events are folded into `state/live.ts`'s per-tab `LiveTab` object as the transcript streams in — `todos: TodoItem[]` (the current list, each item a `content` / `status` / `activeForm` triple, status one of `pending` / `in_progress` / `completed`) and `plans: PlanEntry[]` (a ring buffer of up to 50 revisions, newest first, each an `{ at, content }` pair). This is why Plans & Tasks tracking is read-only in the app: you're looking at a live mirror of what the agent already committed to, not a UI you edit.

Where you actually see it: the footer to-do chip

The one place tasks surface today is the Almanac footer at the bottom of the window — a small `✓ N` / `● N` / `○ N` chip (completed / in-progress / pending) that only appears once the active session's session has called `TodoWrite` at least once. Click it to expand the full checklist in a popover: completed items are struck through, the in-progress item shows its `activeForm` text with a pulsing dot, everything else sits dim and unchecked.

> This chip replaced an earlier, dedicated "Tasks" tab in the left nav. There is no standalone Tasks screen any more — the footer chip is scoped to whichever tab is active, so switching tabs switches whose checklist you're looking at.

figure pending capture: Almanac footer, todo chip expanded, mixed completed/in-progress/pending items

1 Counts at a glance — done / active / waiting. 2 Click to expand; click outside to close. 3 The in-progress row shows the agent's own `activeForm` wording, not the static task text.

Plans and tasks inside a Session's Discussion feed

Every `todo_write` and `plan` event is also a turn in the Session's chat transcript, rendered as a compact tool-family chip — the same chip style as any other tool call (labelled **TodoWrite** or **Plan**). This is governed by the same five-level verbosity dial that controls the rest of the Discussion feed (`lib/chatVerbosity.ts`): tool-family events, todo updates and plan revisions included, only render at dial levels **1 · raw** and **2 · detail**. Turn the dial up to **3 · standard** or quieter and they disappear from the feed entirely — you're left with the interactive back-and-forth, not the bookkeeping.

That's a deliberate trade, not a bug: if you want to audit exactly when the plan changed or the checklist got rewritten, drop to *detail*. If you're skimming a long-running Session for what the agent said and asked, stay at *standard* or above and let the chips stay hidden.

The common mistake

Treating the to-do chip as a place to track work *you* assign. It isn't — it's a readout of what the agent itself decided to track via `TodoWrite` . If a Session never calls `TodoWrite` , the chip never appears, no matter how much work is actually happening; conversely, an agent that keeps its own list current gives you the chip for free. You don't drive this surface, the agent does — the lever you actually have is asking the agent (in your prompt) to track its steps, not editing a checklist in the app.

The second mistake is expecting a dedicated Plans browser. The codebase does contain a plan history viewer (a revision list with diff-against-previous), but as of this edition it isn't wired into any navigation tab — nothing in the app links to it. Plan revisions are still captured and still visible as chips in the Discussion feed at *raw*/*detail*, they're just not browsable as a standalone timeline yet.

The workflow that pays off

1. When you kick off a Session with real multi-step scope, ask for a to-do list explicitly in the opening prompt (or trust that Claude Code's own habits will

produce one on nontrivial work) — that's what makes the footer chip populate in the first place.

2. Glance at the chip's counts, not the popover, for the common case: `✓3 ●1 ○2` tells you the agent is mid-flight on one item with two queued, without opening anything.
3. Open the popover only when you need the actual wording — e.g. deciding whether the in-progress item matches what you expected the agent to be doing right now.
4. If you need the audit trail (what changed, when, in what order), switch the Session's Discussion feed to *detail* or *raw* rather than hunting for a separate log — the `TodoWrite` / `Plan` chips are already there, in order, with the rest of the session's activity around them for context.
5. Don't fight the read-only nature of it. If the agent's plan is wrong, say so in your next message — that's a new `ExitPlanMode` / `TodoWrite` call, and the chip and the feed both pick it up automatically.

> One session, one checklist. Because `todos` is overwritten (not appended) on every `TodoWrite` call, the chip always shows the agent's *current* plan for that Session — there's nothing to reconcile between the app's view and the agent's actual state, because they're the same read.

Simple Mode — the chrome-free single-terminal cockpit

Everything else in this manual is about the full app: tabs, Sessions, the Scheduler, two dozen config surfaces. Simple Mode throws all of that away on purpose. It boots one terminal running `claude --dangerously-skip-permissions` in the directory you launched from, and nothing else.

1. What it actually is

Add `--simple` to the launch command:

```
npx claude-code-session-manager@latest --simple
```

`bin/cli.cjs` doesn't do anything special with the flag — it forwards `process.argv.slice(2)` straight through to the spawned Electron binary, the same way every other CLI argument reaches the app. The main process reads it back on an IPC handler, `app:launch-mode`:

```
ipcMain.handle('app:launch-mode', () => ({
  simple: process.argv.includes('--simple'),
  cwd: process.cwd(),
}));
```

`cwd` is whatever directory your shell was sitting in when you ran the command — `cli.cjs` never changes it, so it's inherited straight through to Electron. The renderer calls this once on mount (`window.api.app.launchMode()`, wired in `src/preload/index.cjs`) and, if `simple` comes back true, `App.tsx` takes an early return before any of the normal boot path runs:

```
if (simpleMode === true) return <SimpleShell />
```

> **There is no ambiguity about scope here.** The flag is read once, at boot, from `process.argv`. There is no in-app toggle that flips a running window into Simple Mode or back — you choose it on the command line, or you don't.

2. What's missing, and why that's the point

`SimpleShell.tsx` renders exactly four things: the privacy recording banner, a thin one-line status strip showing the launch directory, one full-window `Terminal`, and the toast layer. No left nav, no tab bar, no New Session card, no Scheduler, no Settings — none of the surfaces this manual otherwise spends most of its pages on.

Present	Absent
One live terminal running `claude --dangerously-skip-permissions`	Left nav, tab bar, all Configure/Observability tabs
`RecordingStatus` — the privacy banner still mounts here	Sessions, Scheduler, PRDs, queue, history
`Toast` — errors still surface	Any persisted multi-tab state from a previous normal-mode session

> **The common mistake:** launching with `--simple` and then going looking for the Scheduler tab, or expecting a New Session button to queue a PRD. None of it is there. Simple Mode isn't a smaller version of the full app's UI with some panels collapsed — it's a different component tree entirely (`SimpleShell` , not the normal `App.tsx` layout), and it was built to have nothing to click except the terminal.

figure pending capture: app launched with --simple, single terminal filling the window

1 The one-line status strip — "simple mode" plus the launch directory, nothing else. 2

The terminal fills the rest of the window. 3 No left nav, no tab bar, no Almanac footer.

3. How the session gets started

Simple Mode doesn't reuse a saved preset selection or wait for you to press anything — it spawns one session immediately, from the same preset the full app calls `sm-dangerous` :

1. `App.tsx` takes `DEFAULT_PRESETS[0]` — the `claude --dangerously-skip-permissions --session-id <id>` preset — regardless of which mode you're in.
2. When `launchMode()` resolves with `simple: true`, it calls a live-spawn path (`spawnLiveTabInCwd`) that creates one tab in `status: 'spawning'` rooted at the launch `cwd`, and skips straight to running the command — there's no dormant tab waiting for you to click into it.
3. Persisted-tab hydration (`hydrateSessions()`) — the step that restores your normal app's tabs from a previous run — is skipped entirely in this path. Simple Mode never shows you yesterday's tabs; it starts one fresh session every time.

Practically: every `--simple` launch is a brand-new `claude` session in whatever directory you ran the command from, with permission prompts already skipped. There's no picker, no resume, no session history to browse — if you want a specific project or a resumed session, `cd` there first, or don't use `--simple`.

4. When to reach for it vs. the full app

Situation	Use
You want the observability/config cockpit — Sessions, Scheduler running PRDs overnight, Settings, Permissions, Agent Library, and so on	The full app: `npx claude-code-session-manager@latest`
You just want `claude --dangerously-skip-permissions` in the current directory, as fast as possible, with none of the app's other surfaces in the way	`--simple`
You're demoing or screen-sharing the terminal and don't want the rest of the UI in frame	`--simple`
You need to queue work to run unattended (Scheduler, PRDs) or track more than one Session in the same project	The full app — Simple Mode has no queue and no second session

> Simple Mode still honors the same privacy invariant as the full app — `RecordingStatus` is mounted unconditionally, so if voice recording is active the banner shows here too. That's the one piece of chrome that's never optional, in either mode.

5. What it isn't

Simple Mode is not a lighter-weight settings profile, not a "safe mode" for troubleshooting the full app, and not a separate installation — it's the same npm package, the same Electron binary, booting a different renderer component tree based on one command-line flag. Nothing about your `~/.claude/` config, skills, or MCP servers changes between the two modes; the CLI session itself behaves identically either way. The only thing Simple Mode strips out is Session Manager's own UI around it.
